

XSD

Introducción

XML Schema es un lenguaje que se utiliza para **validar documentos XML**, verificando que cumplen con unos requisitos concretos y para asegurar que el documento es válido y lo seguirá siendo en el futuro.

Su nombre técnico es **XML Schema Definition (XSD)**, pero su uso es menos frecuente. En cualquier caso, las referencias a **XML Schema** y a **XML Schema Definition (XSD)** suelen ser al mismo concepto.

Un XSD permite especificar los siguientes aspectos de un documento XML:

- La jerarquía de los elementos y los atributos permitidos.
- Las restricciones y las reglas de validación para el contenido de los elementos y atributos.
- La descripción de estructuras y tipos de datos complejos.
- La definición de relaciones entre elementos.

Al igual que ocurre con los documentos XML y los documentos DTD, los documentos XSD son **ficheros de texto plano**, es decir, se pueden crear y modificar con cualquier editor de texto. Deben tener la extensión **.xsd**.

El uso de XSD es **opcional** pero es **recomendable** para garantizar la **interoperabilidad** de los documentos XML entre diferentes sistemas y aplicaciones, ya que proporciona una forma estandarizada y más avanzada de describir la estructura de un documento XML.

Ejemplo básico

Para entender mejor para qué nos sirve un XML Schema, consideremos el siguiente ejemplo.

Imaginemos que queremos definir un documento XML para recoger los datos de los alumnos de un centro educativo. Nos puede interesar recoger su nombre, apellidos (quizá solo el primero sea obligatorio, ya que no todas las personas tienen dos apellidos), fecha de nacimiento, y el curso en el que están matriculados. Además, en algunos casos, puede que queramos recoger la información de su tutor legal (solo en el caso de que el alumno sea menor de edad). Puede, también, que queramos almacenar su dirección postal, aunque no todas las direcciones tienen un piso, o letra, por lo que este dato no sería obligatorio. Además, la fecha de nacimiento debería tener un formato específico (por ejemplo, dd/mm/aaaa). Por último, quizá queremos recoger el número de teléfono, pero este dato no es obligatorio, y, además, podría disponer de varios teléfonos asociados; además, el teléfono debe tener un formato específico (9 dígitos) Todas estas reglas las podremos definir en un XML Schema, de manera que cualquier documento XML que queramos crear para almacenar los datos de un alumno deberá cumplirlas para ser considerado válido.

Ejemplo de un XML Schema que define las reglas anteriores:

alumno.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="alumno">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="nombre" type="xs:string"/>
        <xs:element name="apellido1" type="xs:string"/>
        <xs:element name="apellido2" type="xs:string" minOccurs="0"/>
        <xs:element name="fechaNacimiento" type="xs:date"/>
        <xs:element name="curso" type="xs:string"/>
        <xs:element name="tutorLegal" type="xs:string" minOccurs="0"/>
        <xs:element name="direccion">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="calle" type="xs:string"/>
              <xs:element name="numero" type="xs:string"/>
              <xs:element name="piso" type="xs:string" minOccurs="0"/>
              <xs:element name="letra" type="xs:string" minOccurs="0"/>
              <xs:element name="ciudad" type="xs:string"/>
              <xs:element name="codigoPostal" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```

        </xs:complexType>
    </xs:element>
    <xs:element name="telefono" minOccurs="0" maxOccurs="unbounded" type="xs:string">
        <xs:simpleType>
            <xs:restriction base="xs:string">
                <xs:pattern value="[0-9]{9}" />
            </xs:restriction>
        </xs:simpleType>
    </xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

Ejemplo de un documento XML válido según el XSD anterior:

alumno.xml

```

<alumno
  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
  xs:noNamespaceSchemaLocation="alumno.xsd">
  <nombre>Abel</nombre>
  <apellido1>Estévez</apellido1>
  <fechaNacimiento>2008-05-12</fechaNacimiento>
  <curso>2º ESO</curso>
  <tutorLegal>María González</tutorLegal>
  <direccion>
    <calle>Calle Falsa</calle>
    <numero>123</numero>
    <ciudad>Vilagarcía de Arousa</ciudad>
    <codigoPostal>36600</codigoPostal>
  </direccion>
  <telefono>986112233</telefono>
  <telefono>666777888</telefono>
</alumno>

```



Características

XML Schema proporciona una serie de **ventajas sobre DTD** debido a las siguientes características:

- En comparación con un DTD, la **sintaxis es más avanzada** y proporciona un **mayor nivel de detalle**.
- XSD permite **restringir con mucha precisión los valores** que puede tener un elemento o atributo mediante la especificación de tipos de datos y reglas de validación.
- El lenguaje utilizado en un documento XSD **es lenguaje XML**. Por lo tanto, los mismos procesadores que permiten interpretar ficheros XML permitirían procesar ficheros XSD.
- XSD permite la definición de espacios de nombres (*namespaces*).

Declaración

Para realizar una validación de un documento XML utilizando un XSD, debemos realizar dos pasos:

- Crear el XSD y definir la estructura a validar en él.
- Vincular el XSD creado en el documento XML para que se pueda utilizar para su validación.

Creación del XSD

La creación de un XSD también se conoce como la **creación de un esquema**. Los elementos XML que se utilizan para generar un esquema han de pertenecer al **namespace XML Schema**, el cual establece el prefijo xs: para todos ellos.

Un **fichero XSD** base tiene la siguiente estructura:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <!-- El resto de elementos -->  
</xs:schema>
```



Tanto en el documento XML como en el documento XSD, **opcionalmente**, podemos incluir la **declaración XML** al inicio del fichero:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- El resto de elementos -->
</xs:schema>
```



En los ejemplos omitiremos la declaración XML por claridad, pero es recomendable incluirla.

El elemento `schema` siempre será el **elemento raíz** y contiene **declaraciones para todos los elementos y atributos** que puedan aparecer en un documento XML que deseemos validar. Además, tendrá definido el **atributo `xmlns:xs`** con la URL al namespace de XML Schema.

Algunos elementos hijos posibles son:

- `<xs:element>`
- `<xs:attribute>`
- `<xs:simpleType>`
- `<xs:complexType>`
- `<xs:group>`
- `<xs:attributeGroup>`

Cada uno de estos elementos de XML Schema tiene una función específica en la definición de los tipos de datos y las restricciones de un documento XML, como veremos en los siguientes apartados.

Vincular el XSD en el documento XML

Una vez creado el documento XSD, debemos indicar en el documento XML que queremos validarlo siguiendo el XSD definido. Para ello, debemos añadir unos atributos en el elemento raíz del documento XML:



```
<raiz
  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
  xs:noNamespaceSchemaLocation="esquema.xsd">
  <!-- ... -->
</raiz>
```

Donde:

- El atributo `xmlns:xs` **tiene siempre el valor** `http://www.w3.org/2001/XMLSchema-instance`. Es la definición de un *namespace*, el cual se utiliza para indicar que el documento está siendo validado contra un esquema XSD. El sufijo `xs` puede ser otro diferente, pero tenemos que utilizar siempre el mismo a lo largo del documento XML.
- El atributo `xs:noNamespaceSchemaLocation` referencia al **fichero XSD donde se encuentra el esquema**. En este caso, el archivo es `esquema.xsd`, el cual se debe encontrar en el mismo directorio que el documento XML. Se está utilizando el prefijo `xs:`, que forma parte del namespace definido en el atributo anterior. Es decir, se está definiendo un **atributo** `noNamespaceSchemaLocation` que forma parte del espacio de nombres `xs`.

❗ URL DE XML SCHEMA

Algunas aplicaciones no validan correctamente los documentos XML si indicamos las URLs en su versión HTTPS. Es decir, debemos evitar las siguientes URLs:

```
https://www.w3.org/2001/XMLSchema
https://www.w3.org/2001/XMLSchema-instance
```



En su lugar, utilizamos las siguientes:

```
http://www.w3.org/2001/XMLSchema
http://www.w3.org/2001/XMLSchema-instance
```



💡 CREACIÓN DE UN ESPACIO DE NOMBRES

Los **espacios de nombres** se indican en el elemento raíz de un documento XML utilizando el prefijo `xmlns:` (XML Namespace). A continuación del prefijo, se indica un **nombre** para el namespace. El **valor del atributo es una URL** que permite identificarlos de forma única.

Para utilizar el espacio de nombres, únicamente tenemos que añadir como prefijo el nombre elegido para el namespace a un elemento o atributo.

```
<cartas
  xmlns:baraja="https://www.lmsgi.com/baraja"
  xmlns:restaurante="https://www.lmsgi.com/restaurante">
  <baraja:carta></baraja:carta>
  <restaurante:carta></restaurante:carta>
</cartas>
```



Consideremos el siguiente documento XML:

alumna.xml

```
<alumna
  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
  xs:noNamespaceSchemaLocation="alumna.xsd">
  Olga Velarde Cobo
</alumna>
```



Un XML Schema que **valida** el documento anterior sería:

```
`xml title="alumna.xsd"
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="alumna" type="xs:string"/>
</xs:schema>
```

El XSD anterior contiene un elemento `<xs:element>`. Este elemento indica que el documento XML debe tener un elemento con nombre `alumna` y que su contenido es una cadena de caracteres (*string*).

Tipos de elementos XML

A la hora de trabajar con XML Schema es muy importante tener clara la diferencia entre los dos tipos de elementos que nos podemos encontrar en un documento XML:

- **Simples**: no contienen ni elementos ni atributos.
- **Complejos**: contienen elementos y/o atributos.

A continuación, se exponen varios ejemplos de cada uno de los tipos.

Elementos simples

Un elemento simple es aquel que **solo contiene texto**, esto es, que no contiene ni elementos ni atributos. Es el tipo de elemento más sencillo que existe.

Un ejemplo de elemento simple sería el siguiente: `<titulo>Manzana</titulo>`

Elementos complejos

Un elemento complejo es aquel que contiene uno o más elementos (elementos hijo) y/o atributos. Los elementos hijo son elementos que están dentro de otro elemento (elemento padre).

Esto permite las siguientes combinaciones de elementos hijo y atributos:

Elementos hijo	Atributos
0 (*)	1
0 (*)	Varios
1	0
Varios	0

1	1
1	Varios
Varios	1
Varios	Varios

* *Un elemento que no contiene elementos puede tener contenido (texto) o no, es decir, puede ser tanto un elemento con texto como un elemento vacío.*

Elemento con contenido con uno o varios atributos

En el momento en el cual un elemento ya tenga un **atributo**, se vuelve del tipo **complejo**, independientemente de que tenga uno o varios atributos.

Ejemplo de elemento con contenido y 1 atributo:

```
<visitante fecha="2023-01-08">Wendel Yarmouth</visitante>
```



Ejemplo de elemento con contenido y varios atributos:

```
<visitante fecha="2023-01-08" hora="14:49:12">Wendel Yarmouth
```



Elemento vacío con uno o varios atributos

Los elementos vacíos si tienen atributos también son complejos por el hecho de tener atributos. Los elementos vacíos son aquellos que no tienen contenido.

Ejemplo de elemento vacío con 1 atributo:

```
<visita fecha="2023-01-08"/>
```



```
<visita fecha="2023-01-08"></visita>
```



Ejemplo de elemento vacío con varios atributos:

```
<visita fecha="2023-01-08" hora="14:49:12"/>
```



```
<visita fecha="2023-01-08" hora="14:49:12"></visita>
```



Elemento con uno o varios elementos hijos y sin atributos

Ejemplo de elemento con 1 elemento hijo y ningún atributo:

```
<visitante>
  <nombre>Wendel Yarmouth</nombre>
</visitante>
```



Ejemplo de elemento con varios elementos hijo y ningún atributo:

```
<visitante>
  <nombre>Wendel Yarmouth</nombre>
  <ip>56.251.47.97</ip>
</visitante>
```



Elemento con uno o varios elementos hijos y con uno o varios atributos

Ejemplo de elemento con 1 elemento hijo y 1 atributo:

```
<visitante fecha="2023-01-08">
  <nombre>Wendel Yarmouth</nombre>
</visitante>
```



Ejemplo de elemento con varios elementos hijo y 1 atributo:

```
<visitante fecha="2023-01-08">
  <nombre>Wendel Yarmouth</nombre>
  <ip>56.251.47.97</ip>
</visitante>
```



Ejemplo de elemento con 1 elemento hijo y varios atributos:

```
<visitante fecha="2023-01-08" hora="14:49:12">
  <nombre>Wendel Yarmouth</nombre>
</visitante>
```



Ejemplo de elemento con varios elementos hijo y varios atributos:

```
<visitante fecha="2023-01-08" hora="14:49:12">
  <nombre>Wendel Yarmouth</nombre>
  <ip>56.251.47.97</ip>
</visitante>
```



Elementos de XML Schema

Los elementos de XML Schema (XSD) son los **elementos utilizados para describir la estructura de un documento XML**. Estos elementos son los que permiten definir la **estructura, relaciones y restricciones** de un documento XML en un XSD.

Más concretamente, los elementos XSD permiten:

- Especificar qué ****elementos y atributos** deben aparecer en un documento.
- Definir las **restricciones** de elementos y atributos en cuanto a su contenido y orden.
- Especificar los **tipos de datos** para los elementos y atributos.

- **Crear tipos de datos personalizados** para elementos y atributos, permitiendo una validación más precisa.
- Definir **elementos y atributos opcionales y obligatorios**, así como las reglas de **validación** para cada uno de ellos.

Como mínimo, todos los elementos y atributos que se vayan a usar en el ejemplar de un documento XML tienen que declararse en el esquema.

En los siguientes apartados se presenta una lista de los elementos básicos de XSD que están definidos en el estándar de XML Schema.

PREFIJO DEL ESPACIO DE NOMBRES

Aunque los diferentes elementos se presenten por su nombre (sin prefijo), cuando se usan en un esquema, todos ellos van precedidos por el prefijo del namespace de XML Schema.

Algunos ejemplos de documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element />
</xs:schema>
```



```
<xsi:schema xmlns:xsi="http://www.w3.org/2001/XMLSchema">
  <xsi:element />
</xsi:schema>
```



En ambos documentos hay **dos elementos** definidos: **schema** y **element**. La estructura que se está definiendo es la misma. Lo único que cambia es el **prefijo** utilizado: en un caso es **xs** y en otro es **xsi**. Se puede utilizar el prefijo que se desee, aunque debemos asegurarnos de utilizar el mismo prefijo en todo el documento.

En la documentación se presentarán los elementos como **schema** y **element** en lugar de por **xs:schema** y **xs:element**. El motivo de esto, como se acaba de comentar, es porque el prefijo puede variar.

schema

El elemento `schema` es el **elemento raíz** de un XML Schema y contiene la **definición del esquema completo**. Es un elemento **obligatorio** en la definición de un XSD ya que sin él no se puede definir un XML Schema, por lo tanto, siempre será el primer elemento que se debe escribir.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- Elementos XSD -->
</xs:schema>
```



El elemento `schema` tiene un **atributo** `xmlns:xs`, el cual siempre tendrá como valor `http://www.w3.org/2001/XMLSchema`.

element

El elemento `element` se utiliza para declarar un **elemento específico** en un documento XML.

En su versión básica permite definir elementos simples, pero también permite definir **elementos complejos** mediante la combinación de `element` con otros elementos XSD.

La información que permite definir es:

- El nombre del elemento.
- Su tipo de dato.
- El valor por defecto.
- Si tiene un valor fijo y cuál es. Supongamos un documento XML con el siguiente elemento:

```
<divisa>Euro</divisa>
```



El documento XSD que permite validar el código anterior es el siguiente:

divisa.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="divisa" type="xs:string" default="Euro" />
</xs:schema>
```



DOCUMENTO XSD

Recordemos que todo XSD tiene como elemento raíz `schema` y que este elemento contiene todos los demás elementos XSD. En este caso, contiene un único elemento: `element`.

Los atributos que se pueden definir en `element` son:

Atributo	Descripción
<code>name</code>	Nombre del elemento en el documento XML. En el ejemplo anterior, <code>divisa</code> .
<code>type</code>	Tipo de dato. En el ejemplo anterior, <code>divisa</code> se define como <code>string</code> , es decir, cualquier cadena de caracteres . En otro apartado se verán todos los tipos de datos aceptados. De momento, se utilizará siempre <code>string</code> para los ejemplos.
<code>default</code>	Valor predeterminado del elemento.
<code>fixed</code>	Permite asignar un valor fijo al contenido del elemento.

Otras opciones que podemos definir (haciendo uso de otros elementos XSD) son:

- Si es **obligatorio** u **opcional**.
- Su **cardinalidad**, es decir, si puede aparecer una o varias veces en el documento.
- Restricciones adicionales en los valores del elemento. Por ejemplo, un rango de números válidos, un patrón de caracteres, etc.

attribute

El elemento `attribute` se utiliza para declarar un **atributo** de un elemento XML.

La información que permite definir es:

- El **nombre** del atributo.
- Su **tipo de dato**.
- El **valor por defecto**.
- Si tiene un **valor fijo** y cuál es.
- Si es **obligatorio** u **opcional**.

Supongamos un documento XML con el siguiente elemento y su atributo:

```
<ciudad provincia="Lugo">Monforte de Lemos</ciudad>
```



El elemento XSD que permite validar el código anterior es el siguiente:

```
<xs:attribute name="provincia" type="xs:string" default="A Co
```



Los atributos que se pueden definir en `attribute` son:

Atributo	Descripción
<code>name</code>	Nombre del atributo.
<code>type</code>	Tipo de dato permitido como valores del atributo provincia.
<code>default</code>	Valor predeterminado.
<code>use</code>	Con el valor <code>required</code> se indica que se trata de un atributo obligatorio. Si se indica este atributo, no se puede utilizar <code>default</code> de forma simultánea.

fixed

Permite asignar un valor fijo al atributo.

Adicionalmente, haciendo uso de otros elementos XSD, podemos definir **restricciones adicionales** en los valores del atributo.

Dependencia de un elemento XML

Cada vez que se utilice el elemento `attribute` de XML Schema, se deberá incluir un `element`, ya que un atributo siempre va a depender de un elemento. Es decir, **un atributo no existe a no ser que exista previamente un elemento**.

El atributo `provincia` no puede existir si no existe `<ciudad>`:

```
<ciudad provincia="Lugo">Monforte de Lemos</ciudad>
```



El documento XSD que permite validar el XML sería el siguiente:

ciudad.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="ciudad">
    <xs:complexType>
      <xs:simpleContent>
        <xs:extension base="xs:string">
          <xs:attribute name="provincia" type="xs:string" use="required"/>
        </xs:extension>
      </xs:simpleContent>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



En este punto no es necesario entender cómo funciona el código anterior, sino saber que, cuando tenemos que añadir un `attribute`, tendremos que hacer lo mismo con `element`.

Tipos de datos

Los tipos de datos son los **diferentes valores** que puede tomar el atributo type de un elemento XSD. Este atributo representa el tipo de dato que tendrá un elemento o atributo en el documento XML.

Supongamos que tenemos un elemento en un XML Schema como el siguiente:

```
<xs:element name="numero" type="xs:positiveInteger"/>
```



La línea anterior está definiendo que en el documento XML a validar debe haber un elemento `<numero>` y que solo puede tener como contenido un **número entero positivo**. Es decir, el siguiente XML sería válido:

```
<numero>20</numero>
```



Mientras que los siguientes no serían válidos:

```
<numero>-5</numero>  
<numero>3.14</numero>  
<numero>texto</numero>
```



Todos los tipos de datos, cuando se utilizan en un elemento del XML Schema, deben ir junto con el **prefijo del espacio de nombres**. En nuestro caso, utilizamos `xs:`.

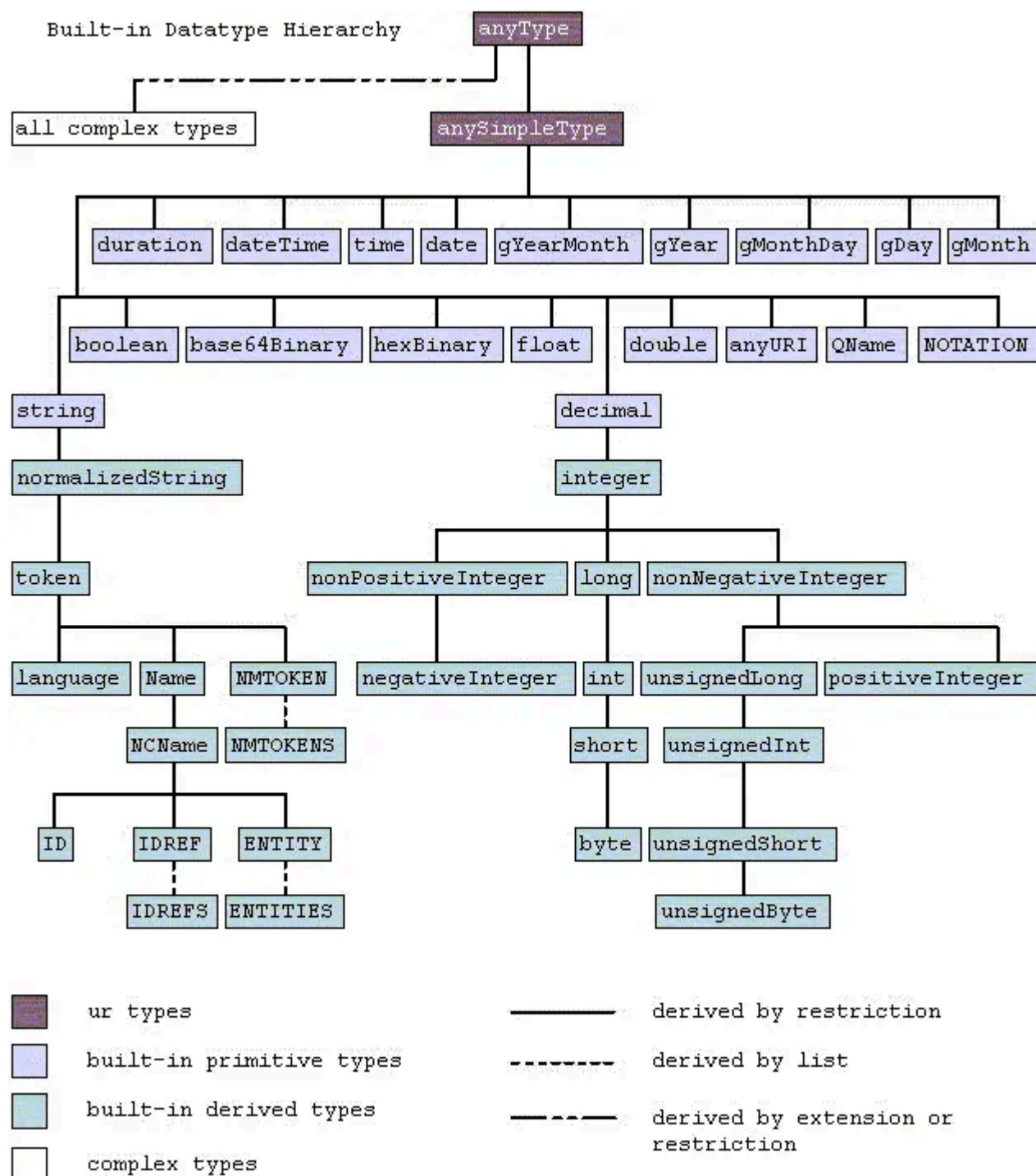
Clasificación de tipos de datos

El estándar de XML Schema divide los tipos de datos en dos grandes grupos:

- **Primitivos:** no se definen a partir de otros tipos de datos.
- **Derivados:** se definen a partir de otros tipos de datos.

En el siguiente diagrama creado por la W3C, se puede observar la jerarquía entre los diferentes tipos. Podemos apreciar que todos los tipos derivados son

creados (derivados) a partir de dos tipos primitivos: string o decimal.



Tipos de datos personalizados

Tenemos dos posibilidades a la hora de definir un tipo de dato para un elemento o atributo:

- Utilizar un tipo de dato **predefinido**, es decir, **proporcionado por el estándar de XML Schema**.

- Utilizar un tipo de dato **personalizado**, es decir, **definido por el programador o programadora**. Este tipo de datos se deben definir previamente y se crean a partir de uno predefinido (de los que se muestran en el diagrama).

En los siguientes apartados se estudiarán algunos de valores del primer grupo, es decir, valores definidos en el estándar que puede tomar el atributo type de los elementos XSD.

Tipos de datos de tipo texto

Algunos de los tipos de datos predefinidos en XML Schema que representan texto son:

- `xs:string`
- `xs:anyURI`
- `xs:language`

Aunque, como se ve en el diagrama anterior, existen muchos tipos derivados de `xs:string` con diferentes aplicaciones:

Tipo de dato	Descripción	Ejemplo
<code>xs:normalizedString</code>	Cadena de caracteres sin saltos de línea, tabulaciones ni espacios múltiples.	"Hola Mundo"
<code>xs:token</code>	Cadena de caracteres sin espacios iniciales o finales, y sin espacios múltiples.	"Hola Mundo"
<code>xs:NMTOKEN</code>	Cadena de caracteres que cumple las reglas de un token XML.	"token_1"
<code>xs:Name</code>	Nombre XML válido.	"nombreElemento"

<code>xs:NCName</code>	Nombre XML válido sin prefijo.	"nombreElemento"
------------------------	--------------------------------	------------------

De momento, no profundizaremos en ellos.

`xs:string`

El tipo de dato `xs:string` se utiliza para representar **cadenas de caracteres**. Es uno de los tipos de datos básicos en XSD y se utiliza para representar cualquier tipo de texto, incluyendo nombres, descripciones, direcciones, etc.

titulo.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="titulo" type="xs:string" />
</xs:schema>
```



Algunos valores válidos para un string son:

```
<titulo>Sofá negro</titulo>
<titulo>Colchones</titulo>
<titulo>Ordenador portátil ultraligero</titulo>
```



`xs:anyURI`

El tipo de dato `xs:anyURI` representa un identificador uniforme de recursos (URI).

Una **URI** (Uniform Resource Identifier) es una cadena de texto que sigue un formato determinado y que permite identificar un recurso de forma inequívoca (sin ambigüedades), como una página web, un archivo o un recurso en un servidor.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="recurso" type="xs:anyURI"/>
</xs:schema>
```



Algunos valores válidos para un anyURI son:

```
<recurso>http://www.w3.org/2001/XMLSchema</recurso>  
<recurso>https://es.wikipedia.org/wiki/Wikipedia</recurso>  
<recurso>/img/logo.png</recurso>  
<recurso>tel:+34666777888</recurso>
```



`xs:language`

El tipo de dato `xs:language` representa un **identificador de un idioma** determinado. Los identificadores vienen definidos en el **RFC 3066**.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="idioma" type="xs:language"/>  
</xs:schema>
```



Algunos valores válidos para un language son:

```
<idioma>es</idioma>  
<idioma>en</idioma>  
<idioma>gl</idioma>  
<idioma>en-us</idioma>  
<idioma>EN-US</idioma>  
<idioma>EN-GB</idioma>  
<idioma>en-GB</idioma>  
<idioma>fr-ca</idioma>
```



Tipos de datos de tipo numérico

Algunos de los tipos de datos predefinidos en XML Schema que representan números son:

- `xs:integer`
- `xs:decimal`
- `xs:float`
- `xs:double`

- `xs:nonNegativeInteger`
- `xs:positiveInteger`

Además, disponemos del tipo de dato `xs:boolean`, que representa valores lógicos (verdadero o falso). Se puede ver en el diagrama anterior todos los tipos derivados de `xs:integer`: `xs:nonPositiveInteger`, `xs:negativeInteger`, `xs:long`, `xs:int`, `xs:short`, `xs:byte`...

`xs:boolean`

El tipo `xs:boolean` permite representar valores **booleanos** (verdadero o falso). Es utilizado para definir elementos o atributos que solo pueden contener valores `true` o `false` (también `1`, equivalente a `true` o `0`, equivalente a `false`).

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="activo" type="xs:boolean"/>
</xs:schema>
```



Algunos valores válidos para un boolean son:

```
<activo>true</activo>
<activo>>false</activo>
<activo>1</activo>
<activo>0</activo>
```



Algunos valores **no válidos** son:

```
<activo>TRUE</activo>
<activo>T</activo>
<activo>F</activo>
<activo>Si</activo>
<activo>Yes</activo>
<activo>No</activo>
```



`xs:integer`

El tipo `xs:integer` se utiliza para representar números **enteros**, es decir, **valores numéricos sin decimales**. Pueden ser **positivos** o **negativos**. Los valores pueden llevar un signo positivo o negativo.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="rango" type="xs:integer"/>
</xs:schema>
```

Algunos valores válidos para un integer son:

```
<rango>3</rango>
<rango>57</rango>
<rango>0</rango>
<rango>-45</rango>
<rango>-1</rango>
<rango>+6</rango>
<rango>0</rango>
```

Algunos valores no válidos son:

```
<rango>3.</rango>
<rango>4.5</rango>
<rango>23.0</rango>
<rango>treinta</rango>
<rango>ZERO</rango>
```

`xs:positiveInteger`

El tipo `xs:positiveInteger` es utilizado para representar un **número entero positivo** (sin signo). Es decir, es un subtipo de integer, lo que significa que tiene las mismas propiedades y restricciones.

El 0 NO se considera entero positivo, sino que el valor mínimo para un `positiveInteger` es 1.



PARA INCLUIR EL 0

Si queremos incluir el 0 como valor válido, debemos utilizar el tipo de dato `xs:nonNegativeInteger`.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="edad" type="xs:positiveInteger"/>
</xs:schema>
```



Algunos valores válidos para un `positiveInteger` son:

```
<edad>3</edad>
<edad>57</edad>
<edad>+6</edad>
```



Algunos valores no válidos son:

```
<edad>0</edad>
<edad>3.</edad>
<edad>-9</edad>
<edad>4.5</edad>
<edad>-23.0</edad>
<edad>four</edad>
<edad>dos</edad>
```



Ejemplos de uso de este tipo son:

- El número de páginas de un libro.
- El número de habitaciones en un hotel.
- El número de ruedas de un vehículo.
- El número de alumnos en una clase.

`xs:negativeInteger` #

El tipo `xs:negativeInteger` es utilizado para representar un **número entero negativo** (signo negativo). Es decir, es un subtipo de integer, lo que significa que tiene las mismas propiedades y restricciones.

El 0 NO se considera entero negativo.

💡 PARA INCLUIR EL 0

Si queremos incluir el 0 como valor válido, debemos utilizar el tipo de dato `xs:nonPositiveInteger`.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="rango" type="xs:negativeInteger"/>
</xs:schema>
```



Algunos valores válidos para un negativeInteger son:

```
<rango>-33</rango>
<rango>-7</rango>
<rango>-1</rango>
<rango>-999</rango>
```



Algunos valores no válidos son:

```
<rango>0</rango>
<rango>3.</rango>
<rango>-4.5</rango>
<rango>6</rango>
<rango>-23.0</rango>
<rango>four</rango>
<rango>dos</rango>
```



`xs:decimal`

El tipo `xs:decimal` se utiliza para representar **números decimales**. Se debe utilizar `.` para la separación de decimales. Los números ser positivos o negativos y, por tanto, llevar un signo positivo o negativo (por omisión, los números son positivos).

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="temperatura" type="xs:decimal"/>
</xs:schema>
```



```
</xs:schema>
```

Algunos valores válidos para un decimal son:

```
<temperatura>3</temperatura>
<temperatura>57.4</temperatura>
<temperatura>8.543233</temperatura>
<temperatura>0</temperatura>
<temperatura>2</temperatura>
<temperatura>2.0</temperatura>
<temperatura>2.000</temperatura>
<temperatura>.66</temperatura>
<temperatura>+80</temperatura>
```



Algunos valores no válidos son:

3,7



DISTINTOS TIPOS DE PRECISIÓN

XSD ofrece tres tipos principales para números decimales:

- **xs:decimal**: Tipo exacto con precisión arbitraria. Ideal para valores financieros o donde se requiere exactitud absoluta. Permite especificar el número de dígitos totales (**totalDigits**) y decimales (**fractionDigits**). No tiene límite de precisión.
- **xs:float**: Número en formato IEEE 754 de precisión simple (32 bits). Utiliza menos memoria pero tiene menor precisión (~7 dígitos significativos). Admite valores como **Infinity**, **-Infinity** y **NaN**. Útil para aplicaciones científicas o cálculos donde la pequeña pérdida de precisión es aceptable.
- **xs:double**: Número en formato IEEE 754 de precisión doble (64 bits). Mayor precisión que **float** (~15 dígitos significativos) pero más memoria. También admite **Infinity**, **-Infinity** y **NaN**. Es el tipo por defecto para números decimales en muchos lenguajes de programación.

Recomendación: Usa `xs:decimal` para datos financieros o contables, y `xs:float`/`xs:double` para cálculos científicos o mediciones físicas.

Tipos de datos de tiempo

Algunos de los tipos de datos predefinidos en XML Schema que representan tiempo son:

- `xs:dateTime`
- `xs:date`
- `xs:time`
- `xs:gYear`
- `xs:gMonth`
- `xs:gDay`
- `xs:gYearMonth`
- `xs:gMonthDay`
- `xs:duration`

`xs:dateTime`

El tipo `xs:dateTime` se utiliza para representar una **fecha y hora combinadas** en el formato **ISO 8601**. Este formato permite especificar tanto la fecha como la hora exacta del día en un solo valor.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="fechaHora" type="xs:dateTime"/>
</xs:schema>
```



El formato es: **YYYY-MM-DDTHH:MM:SS** (donde T es el separador entre la fecha y la hora). Opcionalmente, se puede incluir la zona horaria. El número de dígitos de cada componente es fijo.

Propiedad	Cifras	Valor
-----------	--------	-------

Año	4	0000-9999
Mes	2	01-12
Día	2	01-31
Hora	2	00-23
Minuto	2	00-59
Segundo	2	00-59

Para definir fracciones de segundo, se puede añadir un punto decimal seguido de una o más cifras (por ejemplo, `23:59:59.999`). La fracción puede tener entre 1 y 12 cifras.

Por otra parte, para definir la zona horaria, se puede añadir una Z (para UTC) o un desplazamiento con respecto a UTC en el formato $\pm\text{HH}:\text{MM}$ (por ejemplo, `+02:00` o `-05:00`).

Algunos valores válidos para un `dateTime` son:

```
<fechaHora>2024-12-31T23:59:59</fechaHora>
<fechaHora>2023-01-15T10:30:45</fechaHora>
<fechaHora>2024-06-20T14:25:30.500</fechaHora>
<fechaHora>2024-12-31T23:59:59Z</fechaHora>
<fechaHora>2024-12-31T23:59:59+02:00</fechaHora>
<fechaHora>2024-12-31T23:59:59-05:00</fechaHora>
```



Algunos valores no válidos son:

```
<fechaHora>31/12/2024 23:59:59</fechaHora>
<fechaHora>2024-12-31 23:59:59</fechaHora>
<fechaHora>2024-13-31T23:59:59</fechaHora>
<fechaHora>2024-12-32T25:59:59</fechaHora>
<fechaHora>2024/12/31T23:59:59</fechaHora>
```



Ejemplos de uso de este tipo son:

- Registros de fecha y hora de eventos.
- Timestamps de logs o auditoría.
- Fechas de creación o modificación de registros.

xs:date #

El tipo **xs:date** se utiliza para representar una **fecha** sin la hora. El formato es **YYYY-MM-DD** siguiendo el estándar ISO 8601.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="fechaNacimiento" type="xs:date"/>
</xs:schema>
```

Algunos valores válidos para un date son:

```
<fechaNacimiento>2008-05-12</fechaNacimiento>
<fechaNacimiento>1995-01-25</fechaNacimiento>
<fechaNacimiento>2024-12-31</fechaNacimiento>
<fechaNacimiento>2000-02-29</fechaNacimiento>
<fechaNacimiento>1999-01-01</fechaNacimiento>
```

El tipo **xs:date** también admite la inclusión de una zona horaria al final del valor, utilizando una 'Z' para indicar la hora UTC o un desplazamiento con respecto a UTC en el formato $\pm$ HH:MM (por ejemplo, **+02:00** o **-05:00**).

Algunos valores no válidos son:

```
<fechaNacimiento>12/05/2008</fechaNacimiento>
<fechaNacimiento>05-12-2008</fechaNacimiento>
<fechaNacimiento>2008/05/12</fechaNacimiento>
<fechaNacimiento>2024-13-01</fechaNacimiento>
<fechaNacimiento>2024-12-32</fechaNacimiento>
<fechaNacimiento>2024-02-30</fechaNacimiento>
<fechaNacimiento>2008-05-12Z</fechaNacimiento>
<fechaNacimiento>1995-01-25+02:00</fechaNacimiento>
<fechaNacimiento>2024-12-31-05:00</fechaNacimiento>
```

Ejemplos de uso de este tipo son:

- Fechas de nacimiento de personas.
- Fechas de inicio o fin de proyectos.
- Fechas de vencimiento de documentos.
- Fechas de eventos especiales.

xs:time #

El tipo **xs:time** se utiliza para representar una **hora del día** sin la fecha. El formato es **HH:MM:SS** (horas, minutos y segundos). Opcionalmente, se puede incluir milisegundos y zona horaria.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="horaEntrada" type="xs:time"/>  
</xs:schema>
```



Algunos valores válidos para un time son:

```
<horaEntrada>09:00:00</horaEntrada>  
<horaEntrada>14:30:45</horaEntrada>  
<horaEntrada>23:59:59</horaEntrada>  
<horaEntrada>00:00:00</horaEntrada>  
<horaEntrada>12:45:30.500</horaEntrada>  
<horaEntrada>09:00:00Z</horaEntrada>  
<horaEntrada>14:30:45+02:00</horaEntrada>
```



Algunos valores no válidos son:

```
<horaEntrada>9:00:00</horaEntrada>  
<horaEntrada>09:00</horaEntrada>  
<horaEntrada>25:00:00</horaEntrada>  
<horaEntrada>12:60:30</horaEntrada>  
<horaEntrada>14h 30m 45s</horaEntrada>  
<horaEntrada>14.30.45</horaEntrada>
```



Ejemplos de uso de este tipo son:

- Horas de entrada o salida del trabajo.
- Horarios de clases o eventos.
- Turnos de trabajo.
- Horas de disponibilidad de servicios.

`xs:gYear`

El tipo `xs:gYear` se utiliza para representar un **año gregoriano**. El formato es **YYYY**, donde YYYY es un año de cuatro dígitos. Para años anteriores al 0001, se utiliza un signo negativo, pero siempre con cuatro dígitos (por ejemplo, -0500 para el año 500 a.C.).

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="anoConstitucion" type="xs:gYear"/>
</xs:schema>
```



Algunos valores válidos para un gYear son:

```
<anoConstitucion>2024</anoConstitucion>
<anoConstitucion>2000</anoConstitucion>
<anoConstitucion>1995</anoConstitucion>
<anoConstitucion>-0500</anoConstitucion>
```



Algunos valores no válidos son:

```
<anoConstitucion>24</anoConstitucion>
<anoConstitucion>20240</anoConstitucion>
<anoConstitucion>2024-12</anoConstitucion>
<anoConstitucion>año 2024</anoConstitucion>
```



Ejemplos de uso de este tipo son:

- Año de fundación de una empresa.
- Año de inicio de un proyecto.
- Año de nacimiento cuando no es importante la fecha exacta.

`xs:gMonth`

El tipo `xs:gMonth` se utiliza para representar un **mes gregoriano**. El formato es **--MM** (dos guiones seguidos de dos dígitos para el mes). Siempre hay que indicar los dos guiones, seguido de exactamente dos dígitos.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="mesAniversario" type="xs:gMonth"/>
</xs:schema>
```



Algunos valores válidos para un gMonth son:

```
<mesAniversario>--12</mesAniversario>
<mesAniversario>--01</mesAniversario>
<mesAniversario>--06</mesAniversario>
<mesAniversario>--11</mesAniversario>
```



Algunos valores no válidos son:

```
<mesAniversario>12</mesAniversario>
<mesAniversario>-12</mesAniversario>
<mesAniversario>--13</mesAniversario>
<mesAniversario>--00</mesAniversario>
<mesAniversario>diciembre</mesAniversario>
```



Ejemplos de uso de este tipo son:

- Mes de aniversario.
- Mes de vencimiento de suscripción.
- Mes de celebración recurrente.

`xs:gDay`

El tipo `xs:gDay` se utiliza para representar un **día gregoriano del mes**. El formato es **---DD** (tres guiones seguidos de dos dígitos para el día).




```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="diaFiesta" type="xs:gDay"/>
</xs:schema>
```

Algunos valores válidos para un gDay son:

```
<diaFiesta>---25</diaFiesta>
<diaFiesta>---01</diaFiesta>
<diaFiesta>---15</diaFiesta>
<diaFiesta>---31</diaFiesta>
```



Algunos valores no válidos son:

```
<diaFiesta>25</diaFiesta>
<diaFiesta>--25</diaFiesta>
<diaFiesta>---00</diaFiesta>
<diaFiesta>---32</diaFiesta>
<diaFiesta>veinticinco</diaFiesta>
```



Ejemplos de uso de este tipo son:

- Día del mes para eventos recurrentes.
- Día de la semana o mes del año.

xs:gYearMonth

El tipo **xs:gYearMonth** se utiliza para representar un **año y mes** combinados. El formato es **YYYY-MM**.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="fechaVencimiento" type="xs:gYearMonth"/>
</xs:schema>
```



Algunos valores válidos para un gYearMonth son:



```
<fechaVencimiento>2024-12</fechaVencimiento>
<fechaVencimiento>2025-06</fechaVencimiento>
<fechaVencimiento>1999-01</fechaVencimiento>
<fechaVencimiento>2030-11</fechaVencimiento>
```

Algunos valores no válidos son:

```
<fechaVencimiento>2024-13</fechaVencimiento>
<fechaVencimiento>2024-00</fechaVencimiento>
<fechaVencimiento>24-12</fechaVencimiento>
<fechaVencimiento>2024/12</fechaVencimiento>
<fechaVencimiento>12/2024</fechaVencimiento>
```



Ejemplos de uso de este tipo son:

- Fechas de vencimiento de tarjetas de crédito.
- Períodos de validez de documentos.
- Fechas de expiración de suscripciones (por mes).

`xs:gMonthDay`

El tipo `xs:gMonthDay` se utiliza para representar un **mes y día** sin el año. El formato es **--MM-DD**. Es útil para eventos recurrentes anuales.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="diaEspecial" type="xs:gMonthDay"/>
</xs:schema>
```



Algunos valores válidos para un `gMonthDay` son:

```
<diaEspecial>--12-25</diaEspecial>
<diaEspecial>--01-01</diaEspecial>
<diaEspecial>--06-15</diaEspecial>
<diaEspecial>--02-29</diaEspecial>
```



Algunos valores no válidos son:

```
<diaEspecial>12-25</diaEspecial>
<diaEspecial>-12-25</diaEspecial>
<diaEspecial>--13-25</diaEspecial>
<diaEspecial>--12-32</diaEspecial>
<diaEspecial>--02-30</diaEspecial>
<diaEspecial>25/12</diaEspecial>
```



Ejemplos de uso de este tipo son:

- Fechas de celebraciones (Navidad, Día del Padre, etc.).
- Aniversarios (sin el año).
- Cumpleaños (sin el año).

`xs:duration`

El tipo `xs:duration` se utiliza para representar una **duración** o **intervalo de tiempo**. El formato sigue el estándar ISO 8601:

P[n]Y[n]M[n]DT[n]H[n]M[n]S, donde:

- **P** indica el inicio del período.
- **Y** años, **M** meses, **D** días (después de P).
- **T** separa la fecha de la hora.
- **H** horas, **M** minutos, **S** segundos (después de T).
- **[n]** es un número entero que indica la cantidad de cada unidad de tiempo. Ese número puede ser un número entero o decimal, y puede ser positivo o negativo.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="duracionProyecto" type="xs:duration"/>
</xs:schema>
```



Algunos valores válidos para un duration son:

```
<duracionProyecto>P1Y</duracionProyecto>
<duracionProyecto>P1M</duracionProyecto>
<duracionProyecto>P1D</duracionProyecto>
<duracionProyecto>PT1H</duracionProyecto>
```



```
<duracionProyecto>PT1M</duracionProyecto>
<duracionProyecto>PT1S</duracionProyecto>
<duracionProyecto>P1Y2M3DT4H5M6S</duracionProyecto>
<duracionProyecto>P10D</duracionProyecto>
<duracionProyecto>PT24H</duracionProyecto>
<duracionProyecto>P1Y6M</duracionProyecto>
<duracionProyecto>-P1Y</duracionProyecto>
<duracionProyecto>P-0.27Y</duracionProyecto>
```

que representarían:

- 1 año
- 1 mes
- 1 día
- 1 hora
- 1 minuto
- 1 segundo
- 1 año, 2 meses, 3 días, 4 horas, 5 minutos y 6 segundos
- 10 días
- 24 horas
- 1 año y 6 meses
- -1 año (duración negativa)
- -0.27 años (aproximadamente -3.24 meses)

Algunos valores no válidos son:

```
<duracionProyecto>1 year</duracionProyecto>
<duracionProyecto>1Y2M3D</duracionProyecto>
<duracionProyecto>P1 año</duracionProyecto>
<duracionProyecto>1 hora</duracionProyecto>
<duracionProyecto>P1H</duracionProyecto>
```



FORMATO ISO 8601

La duración sigue el estándar ISO 8601. Algunos ejemplos comunes:

- **P3D** - 3 días

- **PT2H** - 2 horas
- **PT30M** - 30 minutos
- **P1Y** - 1 año
- **P6M** - 6 meses
- **P1Y2M3DT4H5M6S** - 1 año, 2 meses, 3 días, 4 horas, 5 minutos y 6 segundos

Ejemplos de uso de este tipo son:

- Duración de proyectos o tareas.
- Tiempos de espera o procesamiento.
- Duraciones de eventos o cursos.
- Períodos de validez o caducidad.



EJERCICIO RESUELTO

Indica cómo definir las siguientes duraciones en XML Schema utilizando el tipo de dato `xs:duration`:

- 3 días
- 2 horas y 30 minutos
- 1 año, 6 meses y 15 días
- 2 años y 2 segundos

► Solución

Tipos para identificadores #

Los tipos de datos para identificadores en XML Schema permiten definir referencias únicas entre elementos y crear relaciones entre ellos. Estos tipos son especialmente útiles para crear estructuras de datos complejas donde es necesario vincular elementos de manera explícita.

Los principales tipos de identificadores en XML Schema son:

- **xs:ID** - Define un identificador único dentro del documento XML.
- **xs:IDREF** - Define una referencia a un identificador único (de tipo **xs:ID**).
- **xs:IDREFS** - Define múltiples referencias a identificadores únicos (lista de **xs:ID**).
- **xs:ENTITY** - Define una referencia a una entidad XML declarada.
- **xs:ENTITIES** - Define múltiples referencias a entidades XML declaradas.

xs:ID - Identificador único #

El tipo **xs:ID** se utiliza para definir un identificador único dentro de un documento XML. Un elemento o atributo de tipo **xs:ID** debe cumplir las siguientes características:

- El valor debe ser único dentro del documento.
- El valor debe ser un **NCName** válido (nombre sin espacios ni caracteres especiales).
- No puede haber dos elementos o atributos con el mismo valor de tipo **xs:ID**.

Sintaxis:

```
<xs:element name="nombreElemento" type="xs:ID"/>
<xs:attribute name="nombreAtributo" type="xs:ID"/>
```



Ejemplo:

Esquema (empleados.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="empleados">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="empleado" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="nombre" type="xs:string"/>
              <xs:element name="puesto" type="xs:string"/>
              <xs:element name="salario" type="xs:decimal"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



```
        </xs:sequence>
        <xs:attribute name="id" type="xs:ID" use="required"/>
    </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>
```

Documento XML válido (empleados.xml)

```
<empleados>
  <empleado id="emp001">
    <nombre>Juan García</nombre>
    <puesto>Ingeniero de Software</puesto>
    <salario>35000</salario>
  </empleado>
  <empleado id="emp002">
    <nombre>María López</nombre>
    <puesto>Analista de Datos</puesto>
    <salario>32000</salario>
  </empleado>
  <empleado id="emp003">
    <nombre>Carlos Rodríguez</nombre>
    <puesto>Gerente de Proyecto</puesto>
    <salario>40000</salario>
  </empleado>
</empleados>
```

⚠ AVISO

El siguiente documento **NO sería válido** porque hay dos elementos con el mismo **id**:

```
<empleados>
  <empleado id="emp001">
    <nombre>Juan García</nombre>
  </empleado>
  <empleado id="emp001"> <!-- Error: ID duplicado -->
    <nombre>María López</nombre>
```

```
</empleado>
</empleados>
```

El formato de un `xs:ID` debe cumplir las reglas de un NCName: cualquier combinación de letras, dígitos, guiones bajos (`_`), puntos (`.`) y guiones (`-`), pero no puede comenzar con un dígito, guion o punto, ni contener espacios.

`xs:IDREF` - Referencia a un identificador

El tipo `xs:IDREF` se utiliza para crear una referencia a un elemento o atributo de tipo `xs:ID`. El valor de un `xs:IDREF` debe coincidir exactamente con el valor de algún `xs:ID` presente en el documento XML. Es decir, que para que se pueda definir un elemento XML con un atributo o subelemento de tipo `xs:IDREF`, debe existir previamente un elemento o atributo con un valor de tipo `xs:ID` que coincida con ese valor. Las restricciones para crear un IDREF son las mismas que para un ID, excepto la unicidad.

Sintaxis:

```
<xs:element name="nombreElemento" type="xs:IDREF"/>
<xs:attribute name="nombreAtributo" type="xs:IDREF"/>
```



Ejemplo: definimos un elemento `empresa`, que contiene empleados y proyectos. Cada proyecto tiene un responsable que es una referencia a un empleado mediante `xs:IDREF`.

Esquema con referencias (proyectos.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="empresa">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="empleados">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="empleado" maxOccurs="unbounded">
                <xs:complexType>
                  <xs:sequence>
```




```

        <xs:element name="nombre" type="xs:string"/>
      </xs:sequence>
      <xs:attribute name="id" type="xs:ID" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
<xs:element name="proyectos">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="proyecto" maxOccurs="unbounded">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="nombre" type="xs:string"/>
            <xs:element name="responsable" type="xs:IDREF"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

Así, en el siguiente documento XML, cada proyecto tiene un responsable que referencia a un empleado existente mediante su `id`.

Documento XML válido (empresa.xml)

```

<empresa>
  <empleados>
    <empleado id="emp001">
      <nombre>Juan García</nombre>
    </empleado>
    <empleado id="emp002">
      <nombre>María López</nombre>
    </empleado>
  </empleados>
  <proyectos>

```



```

    <proyecto>
      <nombre>Sistema de Facturación</nombre>
      <responsable>emp001</responsable>
    </proyecto>
    <proyecto>
      <nombre>Portal de Análisis</nombre>
      <responsable>emp002</responsable>
    </proyecto>
  </proyectos>
</empresa>

```

Cada proyecto tiene un responsable cuyo valor coincide con el `id` de un empleado definido previamente.

`xs:IDREFS` - Múltiples referencias

El tipo `xs:IDREFS` se utiliza para crear múltiples referencias a elementos o atributos de tipo `xs:ID`. El valor es una lista de identificadores **separados por espacios**.

Sintaxis:

```

<xs:element name="nombreElemento" type="xs:IDREFS"/>
<xs:attribute name="nombreAtributo" type="xs:IDREFS"/>

```



Ejemplo: en un departamento, tenemos empleados y equipos. Cada equipo puede tener múltiples miembros, referenciados mediante `xs:IDREFS`.

Esquema con múltiples referencias (equipos.xsd)

```

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="departamento">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="empleados">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="empleado" maxOccurs="unbounded">
                <xs:complexType>

```



```

        <xs:sequence>
            <xs:element name="nombre" type="xs:string"/>
        </xs:sequence>
        <xs:attribute name="id" type="xs:ID" use="required"/>
    </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
<xs:element name="equipos">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="equipo" maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:element name="nombre" type="xs:string"/>
                        <xs:element name="miembros" type="xs:IDREFS"/>
                    </xs:sequence>
                </xs:complexType>
            </xs:element>
        </xs:sequence>
    </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

Documento XML válido (departamento.xml)

```

<departamento>
  <empleados>
    <empleado id="emp001">
      <nombre>Juan García</nombre>
    </empleado>
    <empleado id="emp002">
      <nombre>María López</nombre>
    </empleado>
    <empleado id="emp003">
      <nombre>Carlos Rodríguez</nombre>
    </empleado>
  </empleados>

```



```

<equipos>
  <equipo>
    <nombre>Equipo Frontend</nombre>
    <miembros>emp001 emp003</miembros>
  </equipo>
  <equipo>
    <nombre>Equipo Backend</nombre>
    <miembros>emp002 emp003</miembros>
  </equipo>
</equipos>
</departamento>

```

Comparación de tipos de identificadores

Tipo	Descripción	Usos	Restricciones
<code>xs:ID</code>	Identificador único	Definir claves primarias	Debe ser único en el documento
<code>xs:IDREF</code>	Referencia única	Crear relaciones uno a uno	Debe referenciar un <code>xs:ID</code> existente
<code>xs:IDREFS</code>	Múltiples referencias	Crear relaciones uno a muchos	Debe referenciar <code>xs:ID</code> existentes

Restricciones

En XML Schema, las restricciones se denominan **facet**as y se refieren a restricciones que podemos aplicar sobre los valores de los datos de un elemento o atributo. Las restricciones se definen mediante el elemento `restriction`:

```

<xs:restriction base="xs:integer">
  <!-- Restricciones -->
</xs:restriction>

```



Donde el atributo `base` es el **tipo de dato base** sobre el cual se aplica la restricción, es decir, partimos de un tipo de dato y vamos acotando la posibilidad de valores que puede tomar. Dicho de otro modo, pasamos de un validación más permisiva a una menos permisiva. En el atributo `base` solo se pueden indicar tipos de datos simples.

Para que se entienda mejor: supongamos que tenemos un elemento que representa la temperatura en grados Celsius, para medir la temperatura corporal humana. Sabemos que la temperatura corporal humana no puede ser menor de 10 grados Celsius, ni mayor a 45 grados Celsius. Por tanto, partimos de un tipo de dato `integer` (entero) y le aplicamos las restricciones de mínimo y máximo.

El elemento `restriction` debe ir dentro de otros elementos XSD y las restricciones se expresan como **elementos hijo**. Veamos un ejemplo con una restricción aplicada:

❗ EJEMPLOS COMPLETOS

En los ejemplos de esta sección se mostrarán los documentos XSD completos, donde se utiliza el elemento `simpleType`, el cual será presentado más adelante. El elemento `simpleType` es necesario definirlo para aplicar restricciones.

En el ejemplo anterior se está partiendo de un tipo `integer`, al cual se le aplica la restricción de que su valor mínimo es 10 y su valor máximo es 45. Es decir, se parte de un tipo de dato que contempla cualquier valor entero a un tipo de dato donde solo se contemplan valores enteros a partir del 10 (incluido) hasta el 45 (incluido).

Algunas de las facetas o restricciones más comunes son:

- `xs:length`
- `xs:minLength`
- `xs:maxLength`
- `xs:enumeration`
- `xs:whiteSpace`

- `xs:minInclusive`
- `xs:maxInclusive`
- `xs:minExclusive`
- `xs:maxExclusive`
- `xs:totalDigits`
- `xs:fractionDigits`
- `xs:pattern`

En los siguientes apartados se explicarán estas restricciones.

`xs:length` - Longitud exacta

La faceta `xs:length` especifica que la longitud de un valor debe ser **exactamente** igual a un número específico de caracteres.

Sintaxis:

```
<xs:length value="número"/>
```



Ejemplo:

Definir un código de producto que debe tener exactamente 6 caracteres:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="codigo">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:length value="6"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- ABC123 (6 caracteres)
- PROD99 (6 caracteres)

Valores inválidos:

- ABC12 (5 caracteres)
- ABC1234 (7 caracteres)

xs:minLength - Longitud mínima

La faceta `xs:minLength` especifica la longitud **mínima** que debe tener un valor. El valor puede tener igual o más caracteres que el especificado.

Sintaxis:

```
<xs:minLength value="número"/>
```



Ejemplo:

Definir una contraseña que debe tener mínimo 8 caracteres:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="contrasena">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:minLength value="8"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- Pass1234 (8 caracteres)
- MiContraseña123 (15 caracteres)

Valores inválidos:

- `Pass123` (7 caracteres)

`xs:maxLength` - Longitud máxima

La faceta `xs:maxLength` especifica la longitud **máxima** que puede tener un valor. El valor puede tener igual o menos caracteres que el especificado.

Sintaxis:

```
<xs:maxLength value="número"/>
```



Ejemplo:

Definir un nombre de usuario que no puede exceder 20 caracteres:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="usuario">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:maxLength value="20"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- `juan` (4 caracteres)
- `juangarciaRodriguez` (19 caracteres)

Valores inválidos:

- `juangarciaRodriguezperez` (24 caracteres)

xs:enumeration - Valores enumerados

La faceta `xs:enumeration` define un **conjunto limitado de valores permitidos**. El elemento solo puede contener uno de los valores especificados.

Sintaxis:

```
<xs:enumeration value="valor1"/>
<xs:enumeration value="valor2"/>
<!-- más valores -->
```



Ejemplo:

Definir un elemento que solo pueda contener los colores permitidos:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="color">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:enumeration value="rojo"/>
        <xs:enumeration value="verde"/>
        <xs:enumeration value="azul"/>
        <xs:enumeration value="amarillo"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Documento XML válido

```
<color>rojo</color>
<color>verde</color>
<color>azul</color>
```



Valores inválidos:

- `<color>naranja</color>` (no está en la enumeración)
- `<color>Rojo</color>` (diferencia de mayúsculas)

`xs:whiteSpace` - Tratamiento de espacios en blanco

La faceta `xs:whiteSpace` controla cómo se tratan los espacios en blanco (espacios, tabulaciones, saltos de línea) en el contenido de un elemento o atributo. Tiene tres valores posibles:

- **preserve** - Se preservan todos los espacios en blanco tal como están. (valor por defecto)
- **replace** - Se reemplazan tabulaciones, saltos de línea y retornos de carro por espacios.
- **collapse** - Se reemplazan tabulaciones, saltos de línea y retornos de carro por espacios, y se eliminan espacios en blanco al inicio y final, además de colapsarse múltiples espacios en uno solo.

Sintaxis:

```
<xs:whiteSpace value="preserve|replace|collapse"/>
```



Ejemplo:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="nombre1">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:whiteSpace value="preserve"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
  <xs:element name="nombre2">
    <xs:simpleType>
      <xs:restriction base="xs:string">
```



```
<xs:whiteSpace value="collapse"/>
</xs:restriction>
</xs:simpleType>
</xs:element>
</xs:schema>
```

Ejemplo de diferencia:

```
<!-- Con preserve: " Juan  García " se mantiene tal cual -->
<nombre1> Juan  García </nombre1>

<!-- Con collapse: " Juan  García " se convierte en "Juan García" --
<nombre2> Juan  García </nombre2>
```

xs:minInclusive - Valor mínimo (incluido)

La faceta `xs:minInclusive` especifica el **valor mínimo permitido, incluyéndolo**. El valor debe ser mayor o igual al especificado.

Sintaxis:

```
<xs:minInclusive value="número"/>
```

Ejemplo:

Definir la edad mínima permitida (mayor o igual a 18):

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="edad">
    <xs:simpleType>
      <xs:restriction base="xs:integer">
        <xs:minInclusive value="18"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

```
</xs:element>
</xs:schema>
```

Valores válidos:

- 18 (igual al mínimo)
- 25
- 100

Valores inválidos:

- 17 (menor que el mínimo)

xs:maxInclusive - Valor máximo (incluido)

La faceta `xs:maxInclusive` especifica el **valor máximo permitido, incluyéndolo**. El valor debe ser menor o igual al especificado.

Sintaxis:

```
<xs:maxInclusive value="número"/>
```



Ejemplo:

Definir la calificación máxima permitida (menor o igual a 10):

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="calificacion">
    <xs:simpleType>
      <xs:restriction base="xs:decimal">
        <xs:maxInclusive value="10"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- 5.5
- 10 (igual al máximo)
- 8.25

Valores inválidos:

- 10.1 (mayor que el máximo)

xs:minExclusive - Valor mínimo (excluido)

La faceta `xs:minExclusive` especifica el **valor mínimo permitido, excluyéndolo**. El valor debe ser estrictamente mayor que el especificado.

Sintaxis:

```
<xs:minExclusive value="número"/>
```



Ejemplo:

Definir que la temperatura debe ser mayor que 0 grados (sin incluir el 0):

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="temperatura">
    <xs:simpleType>
      <xs:restriction base="xs:decimal">
        <xs:minExclusive value="0"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- 0.1

- 25.5
- 100

Valores inválidos:

- 0 (no se incluye el 0)
- -5 (menor que el mínimo)

xs:maxExclusive - Valor máximo (excluido)

La faceta `xs:maxExclusive` especifica el **valor máximo permitido, excluyéndolo**. El valor debe ser estrictamente menor que el especificado.

Sintaxis:

```
<xs:maxExclusive value="número"/>
```



Ejemplo:

Definir que un porcentaje debe ser menor del 100% (sin incluir el 100):

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="porcentaje">
    <xs:simpleType>
      <xs:restriction base="xs:decimal">
        <xs:maxExclusive value="100"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- 50.5
- 99.99

- 0

Valores inválidos:

- 100 (no se incluye el 100)
- 100.1 (mayor que el máximo)

xs:totalDigits - Total de dígitos

La faceta `xs:totalDigits` especifica el **número total de dígitos** permitidos en un número. Se utiliza comúnmente con el tipo `xs:decimal`.

Sintaxis:

```
<xs:totalDigits value="número"/>
```



Ejemplo:

Definir un código de referencia con exactamente 8 dígitos:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="codigoReferencia">
    <xs:simpleType>
      <xs:restriction base="xs:integer">
        <xs:totalDigits value="8"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- 12345678 (8 dígitos)
- 00000001 (8 dígitos)

Valores inválidos:

- 123456789 (9 dígitos)
- 1234567 (7 dígitos)

xs:fractionDigits - Dígitos decimales

La faceta `xs:fractionDigits` especifica el **número máximo de dígitos después del punto decimal**. Se utiliza típicamente con el tipo `xs:decimal`.

Sintaxis:

```
<xs:fractionDigits value="número"/>
```



Ejemplo:

Definir un precio que puede tener máximo 2 dígitos decimales (centavos):

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="precio">
    <xs:simpleType>
      <xs:restriction base="xs:decimal">
        <xs:fractionDigits value="2"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- 19.99
- 100.5
- 50.00
- 25 (sin decimales también es válido)

Valores inválidos:

- 19.999 (3 dígitos decimales)
- 100.123 (3 dígitos decimales)

xs:pattern - Expresión regular

La faceta `xs:pattern` permite especificar un **patrón de expresión regular** que el valor debe cumplir. El patrón sigue la sintaxis de expresiones regulares de XML Schema.

Sintaxis:

```
<xs:pattern value="expresionRegular"/>
```



Ejemplo 1: Formato de teléfono

Definir un teléfono en formato español (9 dígitos):

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="telefono">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:pattern value="[0-9]{9}"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Valores válidos:

- 986112233
- 666777888

Valores inválidos:

- 98611223 (8 dígitos)

- 9861122333 (10 dígitos)
- 98611223a (contiene letras)

Ejemplo 2: Formato de correo electrónico

Definir un correo electrónico con un patrón simple:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="email">  
    <xs:simpleType>  
      <xs:restriction base="xs:string">  
        <xs:pattern value="[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{3,}" />  
      </xs:restriction>  
    </xs:simpleType>  
  </xs:element>  
</xs:schema>
```

Valores válidos:

- usuario@example.com
- juan.garcia@empresa.es

Valores inválidos:

- usuario@.com (falta el dominio)
- usuario@example (falta la extensión)

Ejemplo 3: Código de producto con prefijo

Definir un código que comienza con "PROD" seguido de 4 dígitos:

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="codigoProducto">  
    <xs:simpleType>  
      <xs:restriction base="xs:string">  
        <xs:pattern value="PROD[0-9]{4}" />  
      </xs:restriction>  
    </xs:simpleType>  
  </xs:element>  
</xs:schema>
```

```
<xs:pattern value="PROD[0-9]{4}"/>
</xs:restriction>
</xs:simpleType>
</xs:element>
</xs:schema>
```

Valores válidos:

- PROD0001
- PROD9999

Valores inválidos:

- PROD001 (solo 3 dígitos)
- prod0001 (minúsculas en PROD)



PATRONES COMUNES

Algunos patrones de expresiones regulares útiles:

Patrón	Descripción	Ejemplo
<code>[0-9]{3}-[0-9]{3}-[0-9]{4}</code>	Teléfono con guiones	123-456-7890
<code>[a-z]+@[a-z]+\.[a-z]+</code>	Email simple	user@domain.com
<code>[A-Z]{2}[0-9]{4}</code>	Código: 2 letras + 4 números	AB1234
<code>[0-9]{4}-[0-9]{2}-[0-9]{2}</code>	Fecha YYYY-MM-DD	2025-12-31
<code>[A-Z][a-z]+</code>	Nombre con primera mayúscula	Juan

[a-z0-9_-]+

Nombre de usuario

user_name-123

Expresiones regulares

Una expresión regular es una secuencia de caracteres que se utiliza para definir un patrón. También se conocen como regex (regular expression).

Las expresiones regulares se basan en un conjunto de reglas y símbolos especiales que permiten definir patrones de manera precisa.

En la siguiente tabla se muestran algunas de las expresiones más habituales para construir patrones. En la última columna (Ejemplo), se muestran subrayadas las coincidencias que se obtienen de los patrones de ejemplo. Las palabras que aparecen sin subrayar es que no obtuvieron ninguna coincidencia.

Expresión	Definición	Patrón de ejemplo	Ejemplo
texto	Busca la secuencia de caracteres indicada.	am	programación, Amar, Camarón, Camión
^	Busca el elemento al inicio de una línea.	^Z	Zapato, Zafiro
\$	Busca el elemento al final de una línea.	a\$	María, Silla
*	Busca el elemento anterior a * desde ninguna hasta varias veces.	An*	Aval, Ana, Anna, On Air, Online

+	Busca el elemento anterior a + desde 1 a varias veces.	An+	Ana, Anna
?	Busca el elemento anterior a ? ninguna o una vez.	An?	Aval, Ana
.	Busca cualquier carácter, excepto el de nueva línea. Es un comodín que substituye a un carácter.	A.a	Ana, Asa, Amar
\	El carácter que va después de \ es convertido en carácter especial o, si ya lo es, deja de serlo.	web.es	web.es
[patrón]	Busca cualquier carácter del conjunto indicado entre [y].	[cps]ala	pala, sala, cala
[^patrón]	Busca cualquier carácter que no esté en el conjunto indicado entre [y].	[^AEIOU]	Palanca, Hoy
[a-z]	El guión simboliza un rango.	[a-m]	Palanca, Hoy
{num}	Busca el elemento indicado antes de	a{3}	aaa, baaa

	<code>{num}</code> tantas veces como indique num.		
<code>{min,max}</code>	Busca el elemento indicado antes de <code>{min,max}</code> tantas veces como indique el rango min y max, ambos incluidos.	<code>a{2,3}</code>	aaa, baaa
<code>\b</code>	Busca la palabra exacta, siempre situado en el límite de la palabra, normalmente un espacio.	<code>\bCambia\b</code>	Cambia de página
<code>\d</code>	Busca un dígito del 0 al 9. Tiene el mismo efecto que <code>[0-9]</code> .	<code>\d</code>	8, 9, 567
<code>\D</code>	Busca una coincidencia que no sea un dígito. Tiene el mismo efecto que <code>[^0-9]</code> .	<code>\D</code>	4 páginas, letras
<code>\s</code>	Busca un carácter de espacio (espacio en blanco, tabulador, nueva página, salto de línea, etc.).	<code>\s</code>	espacios en blanco

\w	Busca cualquier carácter de palabra. Tiene el mismo efecto que [a-zA-Z_0-9].	\w	a, 1, _
\W	Busca cualquier carácter que no sea una palabra. Tiene el mismo efecto que [^\w].	\W	?, ¡, !
patrón1 patrón2	Busca el patrón 1 o el patrón 2.	a z	Zoo, Capaz, amar
patrón1(?=patrón2)	Busca el patrón 1 solo si le sigue el patrón 2. El patrón 2 no formará parte de la ocurrencia encontrada.	Hojas (?=pares)	Hojas pares



PRACTICAR EXPRESIONES REGULARES

Para practicar las expresiones regulares, se pueden utilizar las siguientes herramientas online:

- regexr.com
- regex101.com

En ellas, se pueden probar qué ocurrencias se detectan en una cadena de texto a partir de una expresión regular. Además, se ofrece una explicación detallada de la expresión regular introducida.

Las expresiones regulares son válidas para cualquier lenguaje de programación (siempre que soporte esta característica). Se pueden utilizar, incluso, en editores de texto para buscar patrones en el código fuente.

Combinación de restricciones

Es posible **combinar múltiples restricciones** en el mismo elemento. Las restricciones se aplican de forma acumulativa, es decir, el valor debe cumplir **todas** las restricciones especificadas.

Ejemplo:

Definir una contraseña que debe:

- Tener entre 8 y 20 caracteres
- Contener solo letras, números, guiones bajos y guiones

Esquema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="contrasena">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:minLength value="8"/>
        <xs:maxLength value="20"/>
        <xs:pattern value="[a-zA-Z0-9_-] +"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



Documento XML válido

```
<contrasena>MiPass123</contrasena>
<contrasena>user_pass-456</contrasena>
```



Documentos XML inválidos

```
<contrasena>pass</contrasena> <!-- Menos de 8 caracteres -->
<contrasena>MiPassword1234567890123</contrasena> <!-- Más de 20 caract
<contrasena>Mi Pass@123</contrasena> <!-- Contiene espacio y @ (no per
```





EJERCICIO RESUELTO

Define un esquema XSD para validar los siguientes datos:

1. **DNI español:** formato 12345678X (8 dígitos y 1 letra mayúscula)
2. **Código postal español:** formato 28000 (5 dígitos)
3. **Edad:** entre 0 y 150 años
4. **Nombre completo:** mínimo 2 caracteres, máximo 100

► Solución

Definición de tipos de datos

En XSD existen diversos **tipos de datos predefinidos**, los cuales vienen recogidos en el estándar XML Schema. Sin embargo, en algunos casos no es suficiente y es necesario definir **tipos de datos personalizados** que se ajusten mejor a las necesidades específicas de un proyecto.

Para ello, existen diferentes **elementos XSD** que nos ofrecen la posibilidad de definir tipos personalizados. Algunos son:

- simpleType
- complexType
- simpleContent
- complexContent
- extension

La aplicación de **restricciones** para acotar con mayor precisión el tipo de dato que queremos asignar a los diferentes elementos y atributos ya se considera una definición de tipo de dato personalizado, aunque la definición puede ir más allá y tener en cuenta más aspectos, incluyendo tanto elementos de tipo simple como elementos de tipo complejo.

Todo elemento de un XSD debe ser de uno de estos dos tipos:

- **Simple:** no contiene ni elementos ni atributos.
- **Complejo:** contiene elementos y/o atributos.

En esta sección se explicará cómo se crean tipos de datos personalizados, proporcionando ejemplos de su uso en diferentes casos de aplicación.

Tipos de datos simples

Un elemento simple es aquel que solo contiene texto, esto es, que no contiene ni elementos ni atributos.

Por ejemplo:

```
<numero>2</numero>
```



Los elementos simples se pueden declarar en un esquema XSD mediante una línea:

```
<xs:element name="numero" type="xs:integer"/>
```



En la línea anterior, se utiliza como tipo de dato uno **predefinido** en el estándar XML Schema: `integer`.

Si buscamos un tipo de dato que no está recogido en la lista de tipos predefinidos, es cuando entra en juego la **definición de tipos de datos simples** personalizados. Es decir, si queremos indicar al atributo type un valor personalizado. Por ejemplo:

```
<xs:element name="numero" type="numero-par"/>
```



En la línea anterior se establece un tipo `numero-par`. Este tipo no está definido en ningún lado y es trabajo del programador o programadora indicar en qué consiste ese tipo. Para ello, se utilizarán los siguientes elementos XSD (o algunos de ellos):

- `simpleType`

- `restriction`
- `list`
- `union`

`xs:simpleType`

El elemento `simpleType` es un elemento que permite definir un tipo de dato simple personalizado a partir de:

- Un tipo de dato primitivo.
- Un tipo de dato personalizado ya definido (otro `simpleType`).

Se utiliza para **restringir** los valores posibles de un elemento o atributo.

Los tipos de datos personalizados se definen dentro de un elemento `simpleType` y se pueden reutilizar en varios elementos y atributos a lo largo del esquema siempre que se le asigne un valor al atributo `name`.

```
<xs:simpleType name="tipo-simple">
  <!-- Definición del tipo simple -->
</xs:simpleType>
```



El elemento `simpleType` siempre tendrá como elemento hijo alguno de los siguientes:

- `restriction`
- `list`
- `union`

A continuación, veremos el uso con los dos primeros elementos XSD.

`xs:restriction` dentro de `xs:simpleType`

La gran mayoría de las veces, se utiliza `simpleType` con `restriction`, el cual permite definir restricciones en los tipos de datos. Es decir, la mayoría de las veces se utiliza un `simpleType` para definir **tipos de datos más restrictivos** que los proporcionados por el estándar.

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="ciudad">
    <xs:simpleType>
      <xs:restriction base="xs:string">
        <xs:maxLength value="10"/>
      </xs:restriction>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

Podemos observar que define una restricción para el elemento `<ciudad>`. Es decir, se está creando un nuevo tipo donde los valores válidos son cadenas de texto de una **longitud máxima de 10 caracteres**. En el ejemplo anterior, se está aplicando directamente a un elemento.

Creación de un tipo simple

Sin embargo, esto se puede hacer de otra manera:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="ciudad" type="string-10" />

  <xs:simpleType name="string-10">
    <xs:restriction base="xs:string">
      <xs:maxLength value="10"/>
    </xs:restriction>
  </xs:simpleType>
</xs:schema>
```

En el documento XSD anterior, se está declarando un elemento `<ciudad>` donde su tipo de dato es `string-10`. **Este tipo de dato no es un tipo predefinido** en el estándar XML Schema, sino que **lo hemos definido nosotros**. El nombre puede ser cualquiera, siempre que sea una **cadena de caracteres válida para asignar al valor de un atributo XML**.

En las siguientes líneas, podemos ver cómo se define un tipo de dato personalizado. El elemento `simpleType` tiene un atributo `name` con un valor

(string-10). Ese valor es el **nombre** que tomará el nuevo tipo que se definirá. Como se ha comentado, este nombre lo escogemos nosotros.

Ambos documentos XSD presentados son **equivalentes**, con la excepción de que **el último permite reutilizar el tipo de dato creado**. Es decir, si tenemos otro elemento diferente al cual se le quiere aplicar la restricción (o también un atributo), solo basta con indicar el nombre del tipo en su atributo type.

xs:list dentro de xs:simpleType #

El elemento **list** se utiliza para definir una **lista de elementos del mismo tipo**, el cual se define en su atributo **itemType**. Con este elemento, se puede definir una lista de números enteros, una lista de fechas, etc.

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="numeros">
    <xs:simpleType>
      <xs:list itemType="xs:integer"/>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```

En este caso, se define un elemento **<numeros>** que puede contener una serie de números enteros separados por espacios en blanco. Por ejemplo:

```
<numeros>1 2 3 4 5</numeros>
```

Creación de una lista (tipo simple)

De forma alternativa, se podría definir el siguiente esquema:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="numeros" type="lista-enteros" />

  <xs:simpleType name="lista-enteros">
```

```
<xs:list itemType="xs:integer"/>
</xs:simpleType>
</xs:schema>
```

En las líneas anteriores se ha definido el **tipo** `lista-enteros`, el cual se usa en la declaración del elemento `numeros`, pero se podría utilizar el cualquier otro elemento o atributo.

```
<numeros>10 20 30 40 50</numeros>
<numeros>5 15 25</numeros>
```



`xs:union` dentro de `xs:simpleType`

El elemento `union` se utiliza para definir un tipo de dato que puede tomar **valores de varios tipos diferentes**. Es decir, permite combinar varios tipos de datos en uno solo.

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="valor">
    <xs:simpleType>
      <xs:union memberTypes="xs:integer xs:string"/>
    </xs:simpleType>
  </xs:element>
</xs:schema>
```



En este caso, se define un elemento `<valor>` que puede contener tanto un número entero como una cadena de texto. Por ejemplo:

```
<valor>123</valor>
<valor>Hola Mundo</valor>
```



Creación de una unión (tipo simple) #

De forma alternativa, se podría definir el siguiente esquema:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="valor" type="entero-o-texto" />

  <xs:simpleType name="entero-o-texto">
    <xs:union memberTypes="xs:integer xs:string"/>
  </xs:simpleType>
</xs:schema>
```

En las líneas anteriores se ha definido el **tipo entero-o-texto**, el cual se usa en la declaración del elemento **valor**, pero se podría utilizar el cualquier otro elemento o atributo.

```
<valor>456</valor>
<valor>Otro texto</valor>
```

Tipos de datos complejos

Un elemento complejo es aquel que **contiene elementos hijo y/o atributos**.

Algunos ejemplos de elementos complejos son los siguientes:

```
<numero tipo="entero">10</numero>
```

```
<numero tipo="entero" />
```

```
<valores>
  <numero>10</numero>
  <numero>20</numero>
</valores>
```

```
<texto>Esquema <azul>XSD</azul></texto>
```

Todos los datos predefinidos en el estándar son de **tipo simple**. Si es necesario utilizar un tipo complejo, debemos **definirlo previamente en el esquema**.

Los tipos complejos se definen mediante la **combinación de elementos simples y/o estructuras complejas** mediante la utilización de elementos XSD como:

- `complexType`
- `simpleContent`
- `complexContent`
- `extension`
- `sequence`
- `all`
- `choice`
- `group`
- `attributeGroup`

`xs:complexType`

El elemento `complexType` es el elemento base para definir un elemento de tipo complejo.

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="provincia">
    <xs:complexType>
      <xs:simpleContent>
        <xs:extension base="xs:string">
          <xs:attribute name="codigo" type="xs:integer" />
        </xs:extension>
      </xs:simpleContent>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



El esquema anterior define:

- Un elemento llamado `<provincia>` con un contenido de tipo `string` y un atributo `codigo`.

- Un atributo `codigo` obligatorio y del tipo `integer`.

ELEMENTOS XSD

En este punto, no es necesario comprender los elementos `simpleContent` y `extension`. Se presentan en los siguientes apartados.

El código anterior permite validar el siguiente documento XML:

```
<provincia codigo="32">Ourense</provincia>
```



Creación de un tipo complejo

Sin embargo, como ocurre con `simpleType`, esto se puede hacer de otra manera:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="provincia" type="tipo-provincia" />

  <xs:complexType name="tipo-provincia">
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="codigo" type="xs:integer" use="required"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:schema>
```



En las líneas anteriores se define un **tipo de dato complejo personalizado** llamado `tipo-provincia`. Para ello, se separa el elemento `complexType` del elemento `element` y se le añade un atributo `name`. Este tipo se utiliza en la declaración del elemento `<provincia>` indicando el nombre del tipo de dato en el atributo `type`.

Ambos documentos XSD presentados son equivalentes, con la excepción de que el último permite **reutilizar el tipo de dato creado**. Es decir, si tenemos otro elemento diferente al cual se le quiere definir el mismo tipo, solo basta con indicar el nombre del tipo en su atributo `type`.

Contenido de un elemento

El contenido de un elemento se refiere a la **información albergada entre la etiqueta de apertura y cierre** de un elemento.

Por ejemplo, consideremos el siguiente elemento XML:

```
<magnitud>Potencia</magnitud>
```



Su contenido es: `Potencia`

Consideremos este otro elemento XML:

```
<magnitud>
  <electricidad>Potencia</electricidad>
</magnitud>
```



En este caso, el contenido del elemento `<magnitud>` es:

```
<electricidad>Potencia</electricidad>
```

El contenido de un elemento puede ser de tres tipos:

1. **Contenido simple:** solo contiene texto, sin elementos ni atributos.
2. **Contenido complejo:** contiene elementos hijo y/o atributos.
3. **Contenido mixto:** contiene tanto texto como elementos hijo.

Además, un elemento puede no tener contenido, es decir, puede ser un elemento vacío.

Contenido simple

Un elemento con contenido simple es aquel elemento que no contiene otros elementos.

```
<precio divisa="EUR">10.32</precio>
```



Este tipo de elementos se definen con `simpleContent`.

No debemos confundir el contenido del elemento con el tipo de elemento. En el ejemplo anterior, al ser un elemento que contiene un atributo, se trata de un tipo complejo (`complexType`), pero su contenido es simple (`simpleContent`), ya que solo contiene un valor decimal.

Contenido complejo

Un elemento con contenido complejo es aquel elemento que contiene uno o varios elementos.

Por ejemplo.

```
<valores>
  <numero>10</numero>
  <numero>20</numero>
</valores>
```



Este tipo de estructuras se definen con `complexContent`.

En el ejemplo anterior, tenemos un elemento de tipo complejo (`complexType`) con contenido complejo (`complexContent`).

Contenido mixto

Alternativamente, el **contenido también puede ser mixto**, es decir, mezclando cadenas de caracteres con elementos.

```
<texto>Esquema <azul>XSD</azul></texto>
```



En el ejemplo anterior, tenemos un elemento de tipo complejo (`complexType`) con contenido mixto.

El contenido mixto se define mediante el atributo `mixed` del elemento `complexType`. Se debe asignar el valor `true`:

```
<xs:complexType mixed="true">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```



```
<xs:element name="edad" type="xs:positiveInteger"/>
</xs:sequence>
</xs:complexType>
```

Sin contenido

Por último, un elemento puede no tener contenido. En este caso, se le denomina **elemento vacío**. Un elemento vacío se puede expresar de dos formas:

- Mediante una única etiqueta.
- Mediante una etiqueta de apertura y una de cierre sin contenido entre las dos.

Por ejemplo, ambos elementos son equivalentes:

```
<luz/>
<luz></luz>
```



Los elementos vacíos se pueden validar mediante un complexType vacío, sin elementos hijo:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="luz">
    <xs:complexType />
  </xs:element>
</xs:schema>
```



Un elemento vacío puede tener atributos. Por ejemplo:

```
<luz encendida="si" />
<luz encendida="si"></luz>
```



En este caso, solo se define el atributo:

xs:simpleContent #

El elemento `simpleContent` se utiliza para describir el contenido de un elemento cuando el contenido es de un tipo simple (como una cadena de texto, un número, etc.).

Este elemento se puede combinar con las **extensiones**.

⚠ TIPO DE CONTENIDO

Como elemento XSD destinado para el contenido de elementos, no se aplica a atributos.

💡 EXTENSIONES

Una **extensión** es una forma de **agregar atributos o restricciones** a un tipo de datos existente. Se definen mediante **elementos** XSD dentro de extension.

Una extensión se basa en un tipo de datos existente, el cual se especifica en el atributo `base`.

```
<xs:extension base="xs:integer">
  <!-- Reglas -->
</xs:extesion>
```



Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="velocidad" type="magnitud"/>

  <xs:complexType name="magnitud">
    <xs:simpleContent>
      <xs:extension base="xs:decimal">
        <xs:attribute name="unidad" type="xs:string" use="required"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:schema>
```



En el código anterior:

- El elemento hijo de `complexType` es un `simpleContent` ya que el contenido del elemento `<velocidad>` es simple.
- A su vez, se realiza una extensión del tipo decimal (que es el tipo de dato del contenido de `<velocidad>`), añadiendo un nuevo atributo `unidad`.
- El atributo `unidad` es obligatorio y acepta como valores una cadena de texto.

Un documento XML válido sería el siguiente:

```
<velocidad unidad="km/h">60</velocidad>
```



`xs:complexContent`

El elemento `complexContent` se utiliza para describir el contenido de un elemento cuando el contenido es de un **tipo complejo** (contiene otros elementos).

Este elemento se puede utilizar para establecer **restricciones** o para **combinar con extensiones**.

TIPO DE CONTENIDO

Como elemento XSD destinado para el contenido de elementos, no se aplica a atributos.

Restricciones

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="colores" type="lista-colores"/>

  <xs:complexType name="lista-colores">
    <xs:complexContent>
      <xs:restriction base="xs:anyType">
        <xs:sequence>
```



```
        <xs:element name="color" type="xs:string" maxOccurs="unbound" />
    </xs:sequence>
</xs:restriction>
</xs:complexContent>
</xs:complexType>
</xs:schema>
```

En este esquema:

- Se está definiendo un elemento `<colores>` el cual es de tipo `lista-colores`.
- El tipo `lista-colores` es un `complexContent`, es decir, un tipo de elemento que contiene otros elementos.
- Los elementos de `lista-colores` se definen dentro de un elemento `restriction`, el cual debe ser del tipo `xs:anyType`.
- Dentro de `restriction` se define un elemento `sequence` y un elemento `element`. De momento, ignoraremos cómo se utiliza `sequence`. Solo tenemos que saber que si queremos definir un `element`, debe ser dentro de un `sequence`.
- El elemento `element` define un elemento `<color>`, el cual contiene una cadena de caracteres.
- El elemento `<color>` puede aparecer tantas veces como sea necesario, así lo determina el atributo `maxOccurs`, que cuando toma el valor `unbounded` significa que no hay limitación de ocurrencias, es decir, las ocurrencias del elemento son ilimitadas.

El código anterior permite validar un documento XML como el siguiente:

```
<colores>
  <color>Amarillo</color>
  <color>Rojo</color>
  <color>Azul</color>
</colores>
```



De forma alternativa, existe una forma abreviada del documento XSD anterior:



```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="colores" type="lista-colores"/>

  <xs:complexType name="lista-colores">
    <xs:sequence>
      <xs:element name="color" type="xs:string" maxOccurs="unbounded"
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Como se puede observar, los elementos `complexContent` y `restriction` se pueden omitir.

Extensiones

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="fichero" type="info"/>

  <xs:complexType name="info">
    <xs:sequence>
      <xs:element name="nombre" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="extension" type="xs:string"/>
  </xs:complexType>
</xs:schema>
```

El él se está definiendo un elemento `<fichero>` que contiene otro elemento `<nombre>`. Además, `<fichero>` tiene un atributo `extension`. El tipo de dato del elemento `fichero` se está definiendo mediante un `complexType`, al cual se le llama `info`.

El XSD anterior, por ejemplo, permite validar el siguiente documento XML:

```
<fichero extension="xml">
  <nombre>documento.xml</nombre>
</fichero>
```


Si queremos aprovechar el tipo de dato `info` y queremos añadir dos elementos adicionales, lo haríamos de la siguiente manera:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="fichero" type="metadatos"/>

  <xs:complexType name="info">
    <xs:sequence>
      <xs:element name="nombre" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="extension" type="xs:string"/>
  </xs:complexType>

  <xs:complexType name="metadatos">
    <xs:complexContent>
      <xs:extension base="info">
        <xs:sequence>
          <xs:element name="creacion" type="xs:dateTime"/>
          <xs:element name="edicion" type="xs:dateTime"/>
        </xs:sequence>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:schema>
```

Podemos observar que se crea otro `complexType` llamado `metadatos`. Este tipo de dato extiende `info`, es decir, aprovecha lo que hay definido y añade más definiciones. En este caso, se añaden dos nuevos elementos: `<creacion>` y `<edicion>`. Por último, se modifica el valor del atributo `type` del elemento `<fichero>`. De esta forma, se define un nuevo tipo.

Un documento XML que sería validado por el documento anterior sería el siguiente:

```
<fichero extension="xml">
  <nombre>documento.xml</nombre>
  <creacion>2022-12-25T08:00:00</creacion>
  <edicion>2023-01-13T08:00:00</edicion>
</fichero>
```

Podemos observar como el XML es muy similar al anterior presentado: solo se añaden dos nuevos elementos.

Indicadores

Los **indicadores** en XML Schema permiten establecer **cómo se van a escribir (o utilizar) los elementos** en un documento XML. Son elementos XSD que controlan la **estructura y las reglas de validación** de los elementos hijo dentro de un elemento padre.

Los indicadores se pueden clasificar en dos tipos:

- **Indicadores de orden:** permiten definir el **orden** en el que deben aparecer los elementos hijo.
- **Indicadores de ocurrencia:** permiten definir **cuántas veces** puede aparecer un elemento.

Indicadores de orden

Los indicadores de orden permiten definir el **orden** en el que deben aparecer los elementos hijo dentro de un elemento padre. Los principales indicadores de orden en XML Schema son:

- **sequence** (secuencia): los elementos deben aparecer en el **orden exacto** especificado.
- **all** (todos): los elementos pueden aparecer en **cualquier orden**, pero cada uno debe aparecer **como máximo una vez**.
- **choice** (elección): se debe elegir **solo uno de los elementos** disponibles.

xs:sequence - Secuencia ordenada

El elemento **sequence** se utiliza para indicar que los elementos hijo deben aparecer en el **orden exacto** especificado en el esquema. Es el indicador de orden **más restrictivo** y también el **más utilizado**.

Sintaxis:

```
<xs:sequence>
  <xs:element name="elemento1" type="xs:string"/>
```



```
<xs:element name="elemento2" type="xs:integer"/>
<xs:element name="elemento3" type="xs:date"/>
</xs:sequence>
```

Ejemplo:

Definir un esquema para una dirección donde los elementos deben aparecer en un orden específico:

Esquema (direccion.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="direccion">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="calle" type="xs:string"/>
        <xs:element name="numero" type="xs:integer"/>
        <xs:element name="ciudad" type="xs:string"/>
        <xs:element name="codigoPostal" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

También podemos definir el tipo y reutilizarlo:

Esquema (direccion.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="direccion" type="tipo-direccion"/>

  <xs:complexType name="tipo-direccion">
    <xs:sequence>
      <xs:element name="calle" type="xs:string"/>
      <xs:element name="numero" type="xs:integer"/>
      <xs:element name="ciudad" type="xs:string"/>
      <xs:element name="codigoPostal" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

```
</xs:complexType>
</xs:schema>
```

Documento XML válido:

En este caso, los elementos deben aparecer exactamente en el orden especificado: `calle`, `numero`, `ciudad`, `codigoPostal`.

Documento válido (direccion.xml)

```
<direccion>
  <calle>Calle Falsa</calle>
  <numero>123</numero>
  <ciudad>Vilagarcía de Arousa</ciudad>
  <codigoPostal>36600</codigoPostal>
</direccion>
```



Documentos XML inválidos:

Orden incorrecto - INVÁLIDO

```
<direccion>
  <numero>123</numero>
  <calle>Calle Falsa</calle>
  <ciudad>Vilagarcía de Arousa</ciudad>
  <codigoPostal>36600</codigoPostal>
</direccion>
```



Elemento faltante - INVÁLIDO

```
<direccion>
  <calle>Calle Falsa</calle>
  <numero>123</numero>
  <ciudad>Vilagarcía de Arousa</ciudad>
</direccion>
```



Si queremos que uno de los elementos sea opcional, podemos utilizar el atributo `minOccurs`:

Esquema con elemento opcional (direccion.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="direccion" type="tipo-direccion"/>

  <xs:complexType name="tipo-direccion">
    <xs:sequence>
      <xs:element name="calle" type="xs:string"/>
      <xs:element name="numero" type="xs:integer"/>
      <xs:element name="ciudad" type="xs:string"/>
      <xs:element name="codigoPostal" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

Así, el elemento `<codigoPostal>` es opcional y puede estar ausente en el documento XML.

💡 ORDEN EN SEQUENCE

Cuando se utiliza `sequence`:

- Los elementos **deben aparecer en el orden exacto** especificado en el esquema.
- **Todos los elementos** (salvo los opcionales con `minOccurs="0"`) **deben estar presentes**.
- No se pueden omitir elementos intermedios.

`xs:all` - Todos en cualquier orden #

El elemento `all` se utiliza para indicar que los elementos pueden aparecer en **cualquier orden**, pero cada elemento puede aparecer **como máximo una vez**. Este indicador es **menos restrictivo que** `sequence` en cuanto al orden, pero **igualmente restrictivo en cuanto a la cantidad**.

Sintaxis:

```
<xs:all>
  <xs:element name="elemento1" type="xs:string"/>
  <xs:element name="elemento2" type="xs:integer"/>
  <xs:element name="elemento3" type="xs:date"/>
</xs:all>
```



Ejemplo:

Definir un esquema para información de una persona donde los elementos pueden aparecer en cualquier orden:

Esquema (persona.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="persona">
    <xs:complexType>
      <xs:all>
        <xs:element name="nombre" type="xs:string"/>
        <xs:element name="apellido" type="xs:string"/>
        <xs:element name="edad" type="xs:integer"/>
      </xs:all>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



Documentos XML válidos:

Cualquiera de los siguientes documentos sería válido, ya que los elementos pueden aparecer en cualquier orden:

Orden 1 - Válido

```
<persona>
  <nombre>Juan</nombre>
  <apellido>García</apellido>
```



```
<edad>30</edad>
</persona>
```

Orden 2 - Válido

```
<persona>
  <edad>30</edad>
  <nombre>Juan</nombre>
  <apellido>García</apellido>
</persona>
```



Orden 3 - Válido

```
<persona>
  <apellido>García</apellido>
  <edad>30</edad>
  <nombre>Juan</nombre>
</persona>
```



Documento XML inválido:

Elemento duplicado - INVÁLIDO

```
<persona>
  <nombre>Juan</nombre>
  <nombre>Carlos</nombre>
  <apellido>García</apellido>
  <edad>30</edad>
</persona>
```



⚠️ RESTRICCIONES DE ALL

Cuando se utiliza `all`:

- Los elementos pueden aparecer en **cualquier orden**.
- Cada elemento puede aparecer **como máximo una vez** (sin `maxOccurs` o con `maxOccurs="1"`).

- **No se pueden usar elementos con `maxOccurs` mayor que 1** directamente en `all` (aunque algunos esquemas permiten esto con restricciones).
- Los elementos dentro de `all` deben ser **elementos simples**, no otros `complexType` con `sequence` o `choice`.

`xs:choice` - Elegir uno

El elemento `choice` se utiliza para indicar que se debe elegir **solamente uno de los elementos** especificados. No se pueden incluir múltiples elementos diferentes al mismo tiempo.

Sintaxis:

```
<xs:choice>
  <xs:element name="elemento1" type="xs:string"/>
  <xs:element name="elemento2" type="xs:integer"/>
  <xs:element name="elemento3" type="xs:date"/>
</xs:choice>
```



Ejemplo:

Definir un esquema para un producto donde se puede especificar el precio de diferentes formas: en euros, dólares o libras:

Esquema (producto.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="producto">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="nombre" type="xs:string"/>
        <xs:choice>
          <xs:element name="precioEuro" type="xs:decimal"/>
          <xs:element name="precioDolar" type="xs:decimal"/>
          <xs:element name="precioLibra" type="xs:decimal"/>
        </xs:choice>
      </xs:sequence>
    </xs:complexType>
```




```
</xs:element>
</xs:schema>
```

Documentos XML válidos:

Cualquiera de los siguientes documentos sería válido, ya que se ha elegido solo uno de los precios:

Precio en euros - Válido

```
<producto>
  <nombre>Laptop</nombre>
  <precioEuro>999.99</precioEuro>
</producto>
```



Precio en dólares - Válido

```
<producto>
  <nombre>Laptop</nombre>
  <precioDolar>1099.99</precioDolar>
</producto>
```



Precio en libras - Válido

```
<producto>
  <nombre>Laptop</nombre>
  <precioLibra>899.99</precioLibra>
</producto>
```



Documentos XML inválidos:

Múltiples precios - INVÁLIDO

```
<producto>
  <nombre>Laptop</nombre>
  <precioEuro>999.99</precioEuro>
```



```
<precioDolar>1099.99</precioDolar>
</producto>
```

Sin precio - INVÁLIDO

```
<producto>
  <nombre>Laptop</nombre>
</producto>
```



💡 USO DE CHOICE

El elemento `choice` es útil cuando:

- Existe una **alternativa** en la estructura de datos.
- Se necesita elegir **una sola opción** de varias disponibles.
- Se quiere validar documentos con **estructuras alternativas**.

Ejemplos de uso:

- Datos de contacto: elegir entre `email`, `telefono` o `direccion`.
- Métodos de pago: elegir entre `tarjeta`, `transferencia` o `paypal`.
- Tipo de documento: elegir entre `DNI`, `pasaporte` o `licencia de conducir`.

Indicadores de ocurrencia

Los indicadores de ocurrencia permiten definir **cuántas veces** puede (o debe) aparecer un elemento dentro de su elemento padre. Los principales indicadores de ocurrencia en XML Schema son:

- `minOccurs`: especifica el **número mínimo de veces** que debe aparecer un elemento.
- `maxOccurs`: especifica el **número máximo de veces** que puede aparecer un elemento.

`xs:minOccurs` - **Ocurrencia mínima** #

El atributo `minOccurs` especifica el **número mínimo de veces** que un elemento debe aparecer en el documento XML. Si un elemento no especifica `minOccurs`, el valor por defecto es `1`, lo que significa que el elemento es **obligatorio**.

Sintaxis:

```
<xs:element name="elemento" type="xs:string" minOccurs="0"/>
<xs:element name="elemento" type="xs:string" minOccurs="1"/>
<xs:element name="elemento" type="xs:string" minOccurs="2"/>
```

Valores posibles:

- `minOccurs="0"` - El elemento es **opcional** (puede no aparecer).
- `minOccurs="1"` - El elemento es **obligatorio** (debe aparecer al menos una vez). **Este es el valor por defecto.**
- `minOccurs="n"` (donde $n > 1$) - El elemento debe aparecer **mínimo n veces**.

Ejemplo 1: Elemento opcional

Esquema (alumno.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="alumno">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="nombre" type="xs:string" minOccurs="1"/>
        <xs:element name="apellido" type="xs:string" minOccurs="1"/>
        <xs:element name="apellido2" type="xs:string" minOccurs="0"/>
        <xs:element name="telefonoContacto" type="xs:string" minOccurs="0"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

En este esquema:

- `nombre` y `apellido` son **obligatorios** (`minOccurs="1"`).
- `apellido2` y `telefonoContacto` son **opcionales** (`minOccurs="0"`).

Documento XML válido:

Con todos los elementos (alumno1.xml)

```
<alumno>
  <nombre>Juan</nombre>
  <apellido>García</apellido>
  <apellido2>López</apellido2>
  <telefonoContacto>986112233</telefonoContacto>
</alumno>
```



Sin elementos opcionales (alumno2.xml)

```
<alumno>
  <nombre>María</nombre>
  <apellido>Rodríguez</apellido>
</alumno>
```



Documento XML inválido:

Sin elemento obligatorio - INVÁLIDO

```
<alumno>
  <nombre>Carlos</nombre>
</alumno>
```



Ejemplo 2: Mínimo de ocurrencias mayor a 1

Esquema (evento.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="evento">
    <xs:complexType>
      <xs:sequence>
```



```
<xs:element name="nombre" type="xs:string"/>
<xs:element name="fecha" type="xs:date"/>
<xs:element name="asistente" type="xs:string" minOccurs="2"/>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>
```

En este esquema, un evento debe tener **mínimo 2 asistentes**.

Documento XML válido:

Con 3 asistentes - Válido

```
<evento>
  <nombre>Conferencia de Tecnología</nombre>
  <fecha>2025-06-15</fecha>
  <asistente>Juan García</asistente>
  <asistente>María López</asistente>
  <asistente>Carlos Rodríguez</asistente>
</evento>
```



Documento XML inválido:

Con solo 1 asistente - INVÁLIDO

```
<evento>
  <nombre>Conferencia de Tecnología</nombre>
  <fecha>2025-06-15</fecha>
  <asistente>Juan García</asistente>
</evento>
```



`xs:maxOccurs` - Ocurrencia máxima

El atributo `maxOccurs` especifica el **número máximo de veces** que un elemento puede aparecer en el documento XML. Si un elemento no especifica `maxOccurs`, el valor por defecto es `1`, lo que significa que el elemento puede aparecer **como máximo una vez**.

Sintaxis:

```
<xs:element name="elemento" type="xs:string" maxOccurs="1"/>  
<xs:element name="elemento" type="xs:string" maxOccurs="5"/>  
<xs:element name="elemento" type="xs:string" maxOccurs="unbounded"/>
```

Valores posibles:

- `maxOccurs="1"` - El elemento puede aparecer **como máximo una vez**.
Este es el valor por defecto.
- `maxOccurs="n"` (donde $n > 1$) - El elemento puede aparecer **máximo n veces**.
- `maxOccurs="unbounded"` - El elemento puede aparecer **ilimitadas veces** (sin restricción de máximo).

Ejemplo 1: Múltiples ocurrencias limitadas

Esquema (carrito.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="carrito">  
    <xs:complexType>  
      <xs:sequence>  
        <xs:element name="producto" type="xs:string" maxOccurs="10"/>  
      </xs:sequence>  
    </xs:complexType>  
  </xs:element>  
</xs:schema>
```

En este esquema, se pueden agregar **máximo 10 productos** al carrito.

Documento XML válido:

Con 3 productos - Válido

```
<carrito>  
  <producto>Laptop</producto>  
  <producto>Mouse</producto>
```

```
<producto>Teclado</producto>
</carrito>
```

Documento XML inválido:

Con más de 10 productos - INVÁLIDO

```
<carrito>
  <producto>Producto1</producto>
  <!-- ... más productos ... -->
  <producto>Producto11</producto>
</carrito>
```



Ejemplo 2: Ocurrencias ilimitadas

Esquema (pedido.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="pedido">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="cliente" type="xs:string"/>
        <xs:element name="articulo" type="xs:string" maxOccurs="unbound"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```



En este esquema, se pueden agregar **ilimitados artículos** al pedido.

Documento XML válido:

Con muchos artículos - Válido

```
<pedido>
  <cliente>Juan García</cliente>
  <articulo>Artículo 1</articulo>
  <articulo>Artículo 2</articulo>
```



```
<articulo>Artículo 3</articulo>
<articulo>Artículo 4</articulo>
<articulo>Artículo 5</articulo>
<!-- Se pueden añadir más artículos sin límite -->
</pedido>
```

Combinación de `minOccurs` **y** `maxOccurs` **#**

Es muy común combinar `minOccurs` y `maxOccurs` para definir con precisión **cuántas veces** puede aparecer un elemento. Algunos ejemplos comunes son:

minOccurs	maxOccurs	Significado
(no indicado)	(no indicado)	El elemento es obligatorio y aparece exactamente una vez (valores por defecto: 1, 1)
0	1	El elemento es opcional y aparece como máximo una vez
1	1	El elemento es obligatorio y aparece exactamente una vez
0	unbounded	El elemento es opcional y puede aparecer ilimitadas veces
1	unbounded	El elemento es obligatorio y puede aparecer de 1 a ilimitadas veces
2	5	El elemento debe aparecer mínimo 2 veces y máximo 5 veces
3	3	El elemento debe aparecer exactamente 3 veces

Ejemplo práctico:

Esquema (biblioteca.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
  <xs:element name="biblioteca">  
    <xs:complexType>  
      <xs:sequence>  
        <xs:element name="nombre" type="xs:string" minOccurs="1" maxOccurs="1"/>  
        <xs:element name="ciudad" type="xs:string" minOccurs="1" maxOccurs="1"/>  
        <xs:element name="telefono" type="xs:string" minOccurs="0" maxOccurs="3"/>  
        <xs:element name="libro" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>  
      </xs:sequence>  
    </xs:complexType>  
  </xs:element>  
</xs:schema>
```

En este esquema:

- `nombre` y `ciudad` son **obligatorios** (`minOccurs="1"`) y aparecen **exactamente una vez** (`maxOccurs="1"`).
- `telefono` es **opcional** (`minOccurs="0"`) pero si aparece, puede hacerlo **máximo 3 veces** (`maxOccurs="3"`).
- `libro` es **obligatorio** (`minOccurs="1"`) y puede aparecer **ilimitadas veces** (`maxOccurs="unbounded"`).

Documento XML válido:

Biblioteca con 2 teléfonos y varios libros - Válido

```
<biblioteca>  
  <nombre>Biblioteca Municipal</nombre>  
  <ciudad>Madrid</ciudad>  
  <telefono>912345678</telefono>  
  <telefono>912345679</telefono>  
  <libro>Don Quijote</libro>  
  <libro>La Casa de Bernarda Alba</libro>  
  <libro>Cien Años de Soledad</libro>  
  <libro>El Quijote</libro>  
</biblioteca>
```

Documentos XML inválidos:

Sin libros - INVÁLIDO (minOccurs='1' para libro)

```
<biblioteca>
  <nombre>Biblioteca Municipal</nombre>
  <ciudad>Madrid</ciudad>
</biblioteca>
```



Con 4 teléfonos - INVÁLIDO (maxOccurs='3' para telefono)

```
<biblioteca>
  <nombre>Biblioteca Municipal</nombre>
  <ciudad>Madrid</ciudad>
  <telefono>912345678</telefono>
  <telefono>912345679</telefono>
  <telefono>912345680</telefono>
  <telefono>912345681</telefono>
  <libro>Don Quijote</libro>
</biblioteca>
```



EJERCICIO RESUELTO

Define un esquema XSD para un blog que contenga:

1. Un elemento **titulo** obligatorio.
2. Un elemento **autor** obligatorio.
3. Múltiples elementos **comentario** (mínimo 0, máximo 100).
4. Un elemento **etiqueta** que aparezca mínimo 1 vez y máximo 5 veces.

► Solución

Grupos

Los grupos son una forma de agrupar varios elementos y/o atributos y asignarles un nombre común. Esto permite **reutilizar** ese grupo de elementos

y/o atributos en varias partes de un esquema.

Las ventajas de utilizar grupos son varias:

- **Reduce** la cantidad de **código redundante**, ya que permite reutilizar el ya escrito.
- Permite una **mejor estructuración del código**, facilitando la lectura.
- Facilita el **mantenimiento**, ya que al eliminar partes duplicadas, no se pueden producir errores al tener que modificar un valor en varios lugares del esquema.

Grupos de elementos (`xs:group`)

El elemento `group` permite nombrar **agrupaciones de elementos y atributos** para hacer referencia a ellas con un **identificador**. Se utiliza para definir un conjunto de elementos que son comunes en varias partes de un esquema y se pueden reutilizar en lugar de tener que definirlos varias veces.

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="persona" type="datos-personales" />

  <xs:complexType name="datos-personales">
    <xs:sequence>
      <xs:group ref="datos-basicos"/>
      <xs:element name="telefono" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

  <xs:group name="datos-basicos">
    <xs:sequence>
      <xs:element name="nombre" type="xs:string"/>
      <xs:element name="fecha-nacimiento" type="xs:date"/>
    </xs:sequence>
  </xs:group>
</xs:schema>
```

El elemento `group` se puede utilizar para:

- **Definir el grupo.** Para la definición, se utiliza el atributo `name` para asignarle un nombre y se incluyen las reglas en su contenido.
- **Utilizar el grupo.** Para utilizar un grupo definido dentro del esquema, se debe utilizar el atributo `ref` con el nombre del grupo.

En el caso anterior, se está definiendo un grupo `datos-basicos` que incluye una secuencia de dos elementos: `<nombre>` y `<fecha-nacimiento>`. A continuación, se utiliza ese grupo dentro de un tipo de dato llamado `datos-personales`, el cual es utilizado por el elemento `<persona>`.

El tipo `datos-personales` define una secuencia:

- El grupo `datos-basicos`, el cual está formado por una secuencia de dos elementos: `<nombre>` y `<fecha-nacimiento>`.
- El elemento `<telefono>`.

Una definición equivalente a la de `datos-personales` es la siguiente:

```
<xs:complexType name="datos-personales">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="fecha-nacimiento" type="xs:date"/>
    <xs:element name="telefono" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Un ejemplo de documento XML válido es el siguiente:

```
<persona>
  <nombre>Fátima</nombre>
  <fecha-nacimiento>1991-12-31</fecha-nacimiento>
  <telefono>+34 666777888</telefono>
</persona>
```

Grupos de atributos (`xs:attributeGroup`)

El elemento `attributeGroup` permite nombrar **agrupaciones de atributos** para hacer referencia a ellas con un **identificador**.

Consideremos el siguiente documento XSD:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="persona" type="datos-personales" />

  <xs:complexType name="datos-personales">
    <xs:attributeGroup ref="datos-basicos"/>
  </xs:complexType>

  <xs:attributeGroup name="datos-basicos">
    <xs:attribute name="nombre" type="xs:string"/>
    <xs:attribute name="edad" type="xs:positiveInteger"/>
  </xs:attributeGroup>
</xs:schema>
```

El elemento `attributeGroup` se puede utilizar para:

- **Definir el grupo.** Para la definición, se utiliza el atributo `name` para asignarle un nombre y se incluyen las reglas en su contenido.
- **Utilizar el grupo.** Para utilizar un grupo definido dentro del esquema, se debe utilizar el atributo `ref` con el nombre del grupo.

En el caso anterior, se está definiendo un grupo `datos-basicos` que incluye dos atributos: `nombre` y `edad`. A continuación, se utiliza ese grupo de atributos dentro de un tipo de dato llamado `datos-personales`, el cual es utilizado por el elemento `<persona>`.

Un ejemplo de documento XML válido es el siguiente:

Documentación

La documentación de un esquema consiste en añadir **información adicional sobre el esquema**, como **comentarios**, **información de autoría** o **versiones**.

Podemos pensar que un método para añadir esta información es utilizar comentarios. El problema es que los analizadores no garantizan que los comentarios no se modifiquen al procesar los documentos y, por tanto, que los

datos añadidos no se pierdan en algún proceso de transformación del documento.

xs:annotation

En lugar de usar los comentarios, XML Schema tiene definido un elemento `annotation` que permite guardar información adicional. El elemento `annotation` se coloca como **hijo de los elementos a documentar**, como elementos, atributos, tipos de datos y reglas de validación.

Este elemento, a su vez, puede contener una combinación de otros:

- `documentation`
- `appinfo`

xs:documentation

El elemento `documentation` se utiliza para proporcionar información de **documentación humanamente legible** sobre el esquema o una parte específica del esquema. Puede contener elementos XSD y elementos XML.

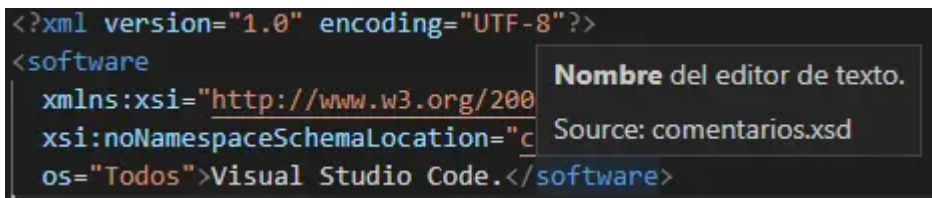
También permite determinar el **idioma** del documento mediante el atributo **xml:lang**.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="software" type="tipo-software">
    <xs:annotation>
      <xs:documentation xml:lang="es-es"><b>Nombre</b> del editor de t
    </xs:annotation>
  </xs:element>

  <xs:complexType name="tipo-software">
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="os" type="xs:string">
          <xs:annotation>
            <xs:documentation xml:lang="es-es">Sistemas operativos sop
          </xs:annotation>
        </xs:attribute>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:schema>
```

```
</xs:extension>
</xs:simpleContent>
</xs:complexType>
</xs:schema>
```

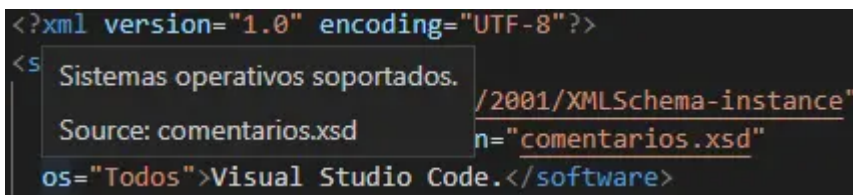
En Visual Studio Code, cuando se pone el cursor encima del elemento `<software>` del documento XML se muestra lo siguiente:



A screenshot of the Visual Studio Code editor. The XML document contains the following code: `<?xml version="1.0" encoding="UTF-8"?>
<software xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="comentarios.xsd" os="Todos">Visual Studio Code.</software>`. A tooltip is displayed over the `<software>` element, showing the text "Nombre del editor de texto." and "Source: comentarios.xsd".

Como se puede observar, se muestra el contenido del elemento `documentation` del XSD asociado al elemento `software`.

Por otro lado, cuando se pone el cursor encima del atributo `os` del documento XML se muestra lo siguiente:



A screenshot of the Visual Studio Code editor. The XML document contains the following code: `<?xml version="1.0" encoding="UTF-8"?>
<software xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="comentarios.xsd" os="Todos">Visual Studio Code.</software>`. A tooltip is displayed over the `os="Todos"` attribute, showing the text "Sistemas operativos soportados." and "Source: comentarios.xsd".

En esta ocasión, como se puede observar, se muestra el contenido del elemento `documentation` del XSD asociado al atributo `os`.

El elemento `documentation` soporta algunos elementos HTML en el contenido.

`xs:appinfo`

El elemento `appinfo` se utiliza para proporcionar **información adicional**. Esta información será utilizada por una aplicación para mostrar una ayuda contextual para elementos y atributos declarados en el esquema.

La información en `appinfo` **no es necesariamente legible para un ser humano** y puede contener instrucciones para una aplicación específica o una herramienta de procesamiento.

Un ejemplo de un XSD documentado sería el siguiente:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="software" type="tipo-software">
    <xs:annotation>
      <xs:appinfo>Nombre del editor de texto.</xs:appinfo>
    </xs:annotation>
  </xs:element>

  <xs:complexType name="tipo-software">
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="os" type="xs:string">
          <xs:annotation>
            <xs:appinfo>Sistemas operativos soportados.</xs:appinfo>
          </xs:annotation>
        </xs:attribute>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:schema>
```

En Visual Studio Code, cuando se pone el cursor encima del elemento `<software>` del documento XML se muestra lo siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<software
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="comentarios.xsd"
  os="Todos">Visual Studio Code.</software>
```

Nombre del editor de texto.
Source: comentarios.xsd

Como se puede observar, se muestra el contenido del elemento `appinfo` del XSD asociado al elemento `software`.

Por otro lado, cuando se pone el cursor encima del atributo `os` del documento XML se muestra lo siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<software
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="comentarios.xsd"
  os="Todos">Visual Studio Code.</software>
```

Sistemas operativos soportados.
Source: comentarios.xsd
n="comentarios.xsd"

En esta ocasión, como se puede observar, se muestra el contenido del elemento `appinfo` del XSD asociado al atributo `os`.

Para que un `appinfo` se muestre en la ayuda contextual de Visual Studio Code, el contenido del elemento debe ser simple (no puede contener otros elementos).

Validación

La validación de XML mediante XML Schema consiste en el proceso de **comparar un documento XML con el esquema XSD**, con el fin de asegurar que el documento XML cumple con las restricciones y estructura especificada en el mismo. Si el documento XML no cumple con las restricciones definidas en el XSD, se genera un error o una advertencia indicando en qué parte del documento se encuentra el error.

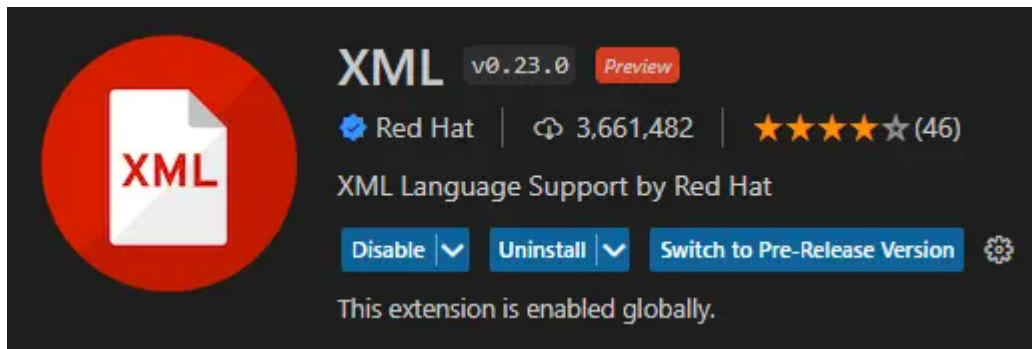
La validación es útil para asegurar la integridad del documento y que se esté utilizando de manera adecuada. También ayuda a detectar errores de formato y estructura en el documento, lo que puede ser útil en el desarrollo y mantenimiento del mismo.

Software

Existen varias aplicaciones para poder validar documentos XML con XML Schema. Muchas de las que permiten validar documentos DTD, también permiten hacer lo mismo con XML Schema.

Visual Studio Code

Visual Studio Code permite validar documentos XML mediante XML Schema con la extensión XML desarrollada por Red Hat.



La extensión XML realiza la validación mientras se escribe el documento o documentos, es decir, no necesitamos pulsar ningún botón para realizar la validación. La validación es continua. Si el documento es válido, no se muestra ningún error:

```
<divisa
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="validar.xsd">EUR</divisa>
```

En el caso de que algo falle, se resaltan los errores:

```
<divisa
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="validar.xsd">EUR</divisa>
```

Para vincular un XSD, debemos añadir los siguientes atributos en la raíz del documento XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<raiz
  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
  xs:noNamespaceSchemaLocation="esquema.xsd">
  <!-- ... -->
</raiz>
```



Liquid Technologies XML Validator

Liquid Technologies XML Validator es un validador XSD online. Es una herramienta útil si no tenemos la posibilidad de instalar ninguna de las anteriores.

Free Online XML Validator (XSD)

Validates an XML document using an XSD schema.

Access the online tools directly from your desktop.
 Download Free Liquid Studio Community Edition Now!

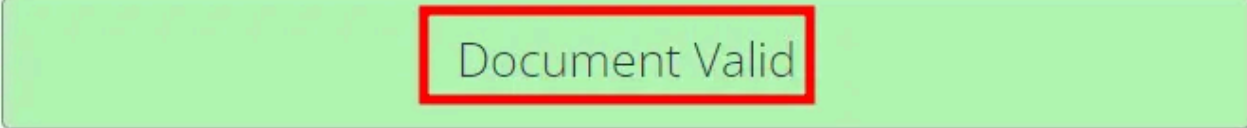
XML data to validate

```
1 <hora>01:24:00</hora>
```

XML schema (XSD) data

```
1 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
2   <xs:element name="hora" type="xs:time"/>  
3 </xs:schema>
```

Validate



Document Valid

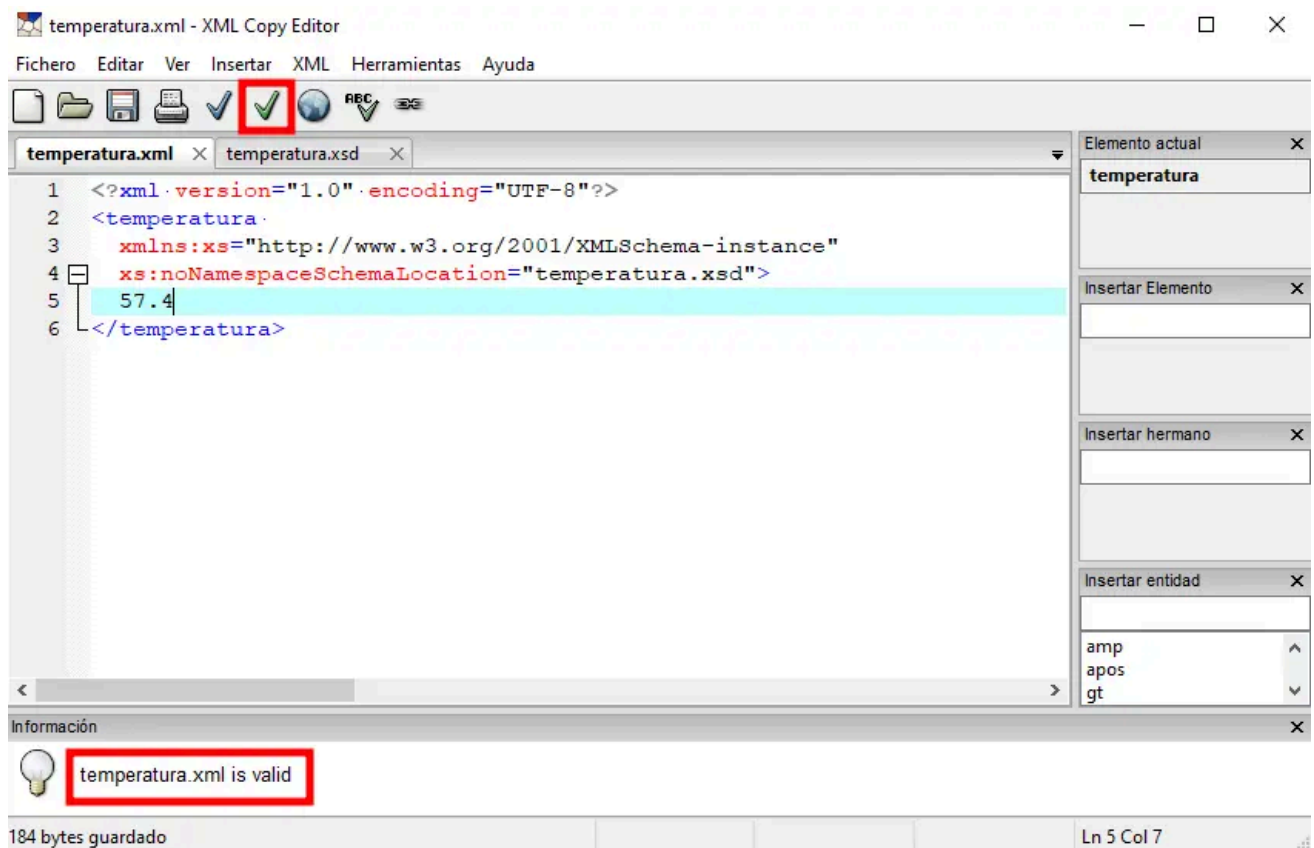
Su uso es muy sencillo. Consiste en los siguientes pasos:

- Copiar el código XML en el área de texto XML data to validate.
- Copiar el el código XSD en el área de texto XML schema (XSD) data.
- Resolver un captcha.
- Pulsar en Validate.

Si el documento XML es válido, se mostrará el mensaje «Document Valid». En caso contrario, se mostrará la lista de errores.

XML Copy Editor

XML Copy Editor es un editor de texto especializado en la edición de documentos XML. Es un software multiplataforma, gratuito, de código abierto que permite validar documentos XML con XSD.



Su uso también es muy sencillo: debemos escribir el documento XML o abrirlo en el editor y pulsar en el icono resaltado en la imagen. El documento XSD debe existir y debe estar correctamente enlazado.

Si el documento XML es válido, se mostrará un mensaje que indica que el documento es válido.